
Los Fundamentos del Analista de Datos

Domina SQL, Excel y Python desde cero

Alex Goyzueta Delgado

alexgoyzueta2018@gmail.com

Los Fundamentos del Analista de Datos

Domina SQL, Excel y Python desde cero

© 2026 Alex Goyzueta Delgado

Todos los derechos reservados. Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en cualquier forma o por cualquier medio, incluyendo fotocopiado, grabación u otros métodos electrónicos o mecánicos, sin el permiso previo por escrito del autor.

Este libro es parte de la colección **Data & Analytics Mastery**.

Para consultas sobre licencias o permisos, contactar a:

alexgoyzueta2018@gmail.com

Primera edición: 2026

ISBN: Pendiente

Editorial: Autoedición

Dedicatoria

A todos los que empiezan hoy.

El camino del dato es un viaje, no un destino.

Que este libro sea tu brújula.

Prefacio

Bienvenido a la Colección Data & Analytics Mastery

Este libro es el primero de una colección de 5 volúmenes diseñada para llevarte desde principiante absoluto hasta profesional especializado en datos. La colección completa es:

1. **Los Fundamentos del Analista de Datos** — SQL, Excel y Python desde cero (este libro)
2. **Python para Datos y tu Primer Data Warehouse Cloud** — Pandas, BigQuery y Looker Studio
3. **Business Intelligence y Modelado de Datos** — Power BI, DAX avanzado y dashboards
4. **Ingeniería de Datos Moderna** — Snowflake, dbt y Airflow
5. **Big Data, ML e IA: Especialización Avanzada** — Spark, streaming y machine learning

Cómo usar este libro

Cada capítulo sigue la misma estructura:

1. **Conceptos teóricos** — Lo que necesitas saber
2. **Ejemplos prácticos** — Código que puedes ejecutar
3. **Ejercicios** — Para practicar (10-15 por capítulo)
4. **Checklist de autoevaluación** — ¿Lo sabes todo?

Al final de cada proyecto hay un **checkpoint** que integra todo lo aprendido.

¿Qué necesitas?

- Un ordenador con Windows, Mac o Linux
- Conexión a internet para descargar los materiales
- Ganas de aprender

Material complementario

Todos los códigos, bases de datos y soluciones están disponibles en:

- Repositorio GitHub: [URL del repositorio]
- Códigos QR al final de cada capítulo

Convenciones tipográficas

`Código en línea` se muestra así.

```
Los bloques de código se muestran así.
```

Negrita para términos importantes.

Cursiva para énfasis.

Sobre la progresión

Este libro cubre los meses 1-3 de formación de un analista de datos. No necesitas experiencia previa. Solo curiosidad y dedicación.

Empecemos.

Introducción

¿Qué es un Analista de Datos?

Un analista de datos es un profesional que recopila, procesa y analiza datos para ayudar a las organizaciones a tomar mejores decisiones. No es necesario ser un matemático ni un programador experto. Un buen analista combina tres habilidades:

1. **Pensamiento crítico** — Hacer las preguntas correctas
2. **Conocimiento técnico** — SQL, Excel, Python
3. **Comunicación** — Contar historias con datos

¿Qué herramientas usarás?

Herramienta	Propósito
SQL	Consultar bases de datos, extraer datos
Excel	Analizar, transformar y visualizar datos
Python	Automatizar procesos y análisis avanzados

El proyecto central: TechStore

A lo largo de este libro trabajarás con los datos de **TechStore**, una tienda de electrónica con presencia en toda España. Ayudarás al equipo de análisis a responder preguntas de negocio reales:

- ¿Qué productos se venden más?
- ¿Quiénes son nuestros mejores clientes?
- ¿En qué temporadas aumentan las ventas?
- ¿Cómo podemos optimizar el inventario?

Cada capítulo construye sobre el anterior. Al final del libro, tendrás un portfolio completo con 4 proyectos reales.

Estructura del libro

Proyecto 1: Explorando los Datos (Capítulos 1-5)
→ Aprenderás lo básico de cada herramienta

Proyecto 2: Análisis de Ventas (Capítulos 6-10)
→ Consultas más complejas, tablas dinámicas

Proyecto 3: Transformación Avanzada (Capítulos 11-15)

→ Power Query, funciones ventana, DAX

Proyecto 4: Automatización (Capítulos 16-18)

→ Integración total: SQL + Python + Excel

Antes de empezar

1. Instala **DB Browser for SQLite** (gratis): <https://sqlitebrowser.org>
2. Asegúrate de tener **Excel** o **LibreOffice Calc** (gratis)
3. Instala **Python 3.12+**: <https://python.org>
4. Instala **VS Code**: <https://code.visualstudio.com>

En el Apéndice C encontrarás una guía detallada de instalación.

Descarga los materiales

Todos los archivos necesarios están en la carpeta `codigos/` de este libro. Incluyen:

- `techstore.db` — Base de datos SQLite
- `datos\_ventas.csv`, `datos\_clientes.csv`, `datos\_productos.csv`
- Scripts Python de ejemplo

¿Listo? Vamos al Capítulo 1.

Capítulo 1: El Ecosistema del Analista de Datos

TechStore: tu primer caso real

TechStore es una cadena de tiendas de electrónica con presencia en toda España. Venden portátiles, smartphones, tablets y accesorios tanto en tiendas físicas como online. Como analista de datos, tu misión será ayudar al equipo a responder preguntas de negocio usando datos reales.

A lo largo de este libro trabajarás con tres herramientas:

- **SQL** para extraer datos de la base de datos TechStore
- **Excel** para analizar y visualizar datos
- **Python** para automatizar y escalar tus análisis

Configuración del entorno

Instalar DB Browser for SQLite

SQLite es una base de datos ligera y autónoma. No necesita servidor. Todo está en un solo archivo.

1. Ve a <https://sqlitebrowser.org>
2. Descarga e instala DB Browser for SQLite
3. Abre el archivo `codigos/techstore.db` que viene con este libro

Instalar Python

1. Ve a <https://python.org/downloads>
2. Descarga Python 3.12 o superior
3. Durante la instalación, marca "Add Python to PATH"
4. Verifica la instalación abriendo una terminal y escribiendo:

```
python --version
```

Instalar VS Code

1. Ve a <https://code.visualstudio.com>
2. Descarga e instala Visual Studio Code
3. Instala las siguientes extensiones:
 - Python (Microsoft)
 - SQLite Viewer

- Excel Viewer

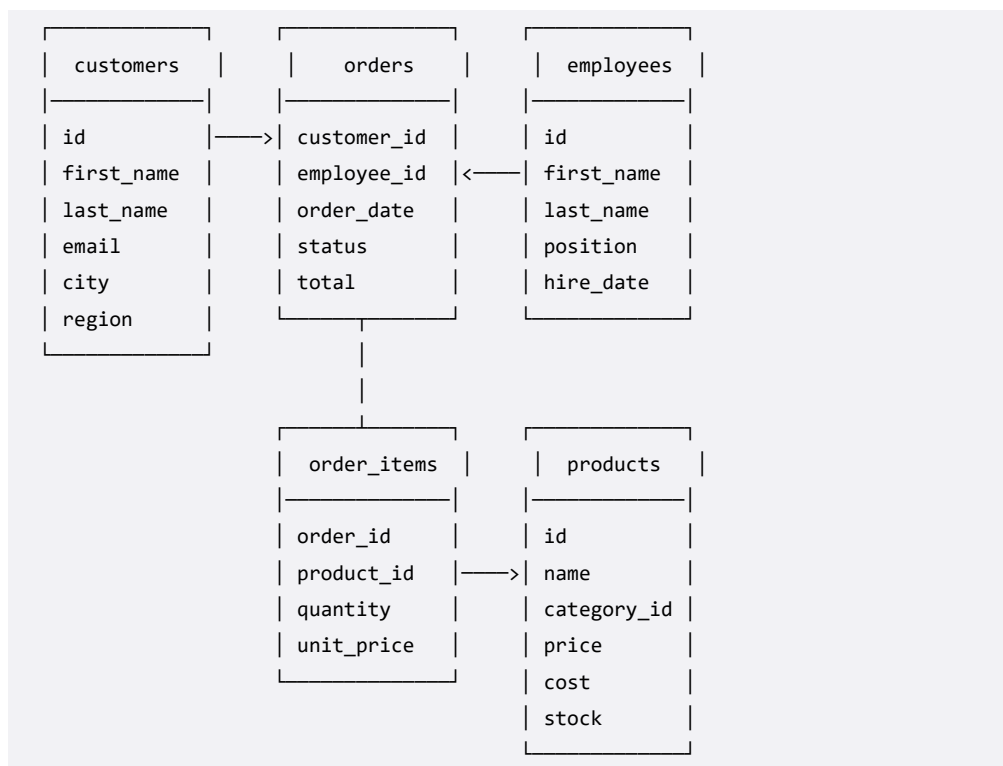
Archivos del libro

Todos los archivos están en la carpeta `codigos/`:

Archivo	Descripción
`techstore.db`	Base de datos SQLite completa
`datos_ventas.csv`	6000 pedidos en formato CSV
`datos_clientes.csv`	250 clientes
`datos_productos.csv`	509 productos con categorías
`datos_detalle_ventas.csv`	21,118 líneas de detalle

Estructura de la base de datos TechStore

La base de datos tiene 5 tablas principales:



Primeros pasos con DB Browser

1. Abre DB Browser for SQLite
2. Haz clic en "Open Database" y selecciona `techstore.db`
3. Ve a la pestaña "Browse Data"
4. Selecciona cada tabla para explorar sus datos

Tómate 5 minutos para explorar las tablas. Mira qué columnas tiene cada una. Familiarízate con los datos.

Ejercicios del Capítulo 1

1. Abre la base de datos TechStore en DB Browser. ¿Cuántas tablas ves?

2. ¿Cuántos registros tiene la tabla `products`?
3. ¿Cuántos clientes hay en la tabla `customers`?
4. ¿Qué columnas tiene la tabla `orders`?
5. ¿Cuál es el rango de fechas de los pedidos? (mira la columna `order\_date`)
6. ¿Cuántos empleados trabajan en TechStore?
7. ¿Cuántas categorías de productos hay?
8. Abre `datos\_ventas.csv` en Excel. ¿Cuántas filas tiene?
9. ¿Cuál es la diferencia entre un archivo `.db` y un archivo `.csv`?
10. ¿Por qué crees que SQLite es más adecuado que Excel para consultar millones de filas?

Checklist de autoevaluación

- ☐ Instalé DB Browser for SQLite y abrí techstore.db
- ☐ Instalé Python 3.12+
- ☐ Instalé VS Code con las extensiones recomendadas
- ☐ Exploré las 5 tablas de la base de datos
- ☐ Abrí los archivos CSV en Excel
- ☐ Entiendo la diferencia entre tablas relacionadas

Capítulo 2: SQL — SELECT y tus Primeros Datos

¿Qué es SQL?

SQL (Structured Query Language) es el lenguaje universal para hablar con bases de datos. Con SQL puedes:

- Extraer datos específicos
- Filtrar registros
- Combinar tablas
- Calcular métricas
- Insertar, actualizar y eliminar datos

Todo analista de datos debe saber SQL. Es la habilidad más demandada en ofertas de empleo de datos.

Tu primera consulta SQL

La sentencia más básica es `SELECT`. Sirve para extraer datos de una tabla.

```
SELECT * FROM products;
```

Esta consulta dice: "dame todas las columnas (`*`) de la tabla `products`".

Pruébalo: Abre DB Browser, ve a la pestaña "Execute SQL", escribe la consulta y haz clic en "Run".

LIMIT: controla cuántos resultados ves

Las tablas pueden tener miles de filas. Usa `LIMIT` para ver solo las primeras:

```
SELECT * FROM products LIMIT 10;
```

Seleccionar columnas específicas

En lugar de `*`, nombra las columnas que quieres:

```
SELECT name, price, stock FROM products LIMIT 10;
```

ORDER BY: ordena los resultados

```
SELECT name, price FROM products ORDER BY price DESC LIMIT 10;
```

`DESC` ordena descendente. `ASC` (o nada) ordena ascendente.

```
SELECT name, price FROM products ORDER BY price ASC LIMIT 10;
```

Ordenar por múltiples columnas

```
SELECT name, category_id, price
FROM products
ORDER BY category_id ASC, price DESC
LIMIT 20;
```

Esto ordena primero por categoría, y dentro de cada categoría, por precio descendente.

Comentarios en SQL

```
-- Esto es un comentario de una línea
SELECT name, price FROM products; -- también al final de una línea

/* Esto es un
   comentario
   de varias líneas */
SELECT * FROM customers;
```

Alias con AS

Usa `AS` para renombrar columnas en los resultados:

```
SELECT
    name AS producto,
    price AS precio,
    stock AS inventario
FROM products
LIMIT 10;
```

Esto hace que los resultados sean más legibles.

Explorando las tablas de TechStore

Prueba estas consultas para familiarizarte con los datos:

```
-- Todos los productos
SELECT * FROM products LIMIT 20;

-- Todos los clientes (solo nombre y ciudad)
SELECT first_name, last_name, city FROM customers LIMIT 20;

-- Todos los pedidos, ordenados del más reciente al más antiguo
SELECT id, customer_id, order_date, total
FROM orders
ORDER BY order_date DESC
LIMIT 20;
```

```
-- Los 5 productos más caros
SELECT name, price FROM products ORDER BY price DESC LIMIT 5;

-- Los 5 productos más baratos
SELECT name, price FROM products ORDER BY price ASC LIMIT 5;
```

Buenas prácticas

1. **Usa mayúsculas para palabras clave SQL:** `SELECT`, `FROM`, `WHERE`
2. **Termina cada consulta con punto y coma (;)**
3. **Usa nombres descriptivos para los alias**
4. **Comienza simple y ve añadiendo complejidad**

Ejercicios del Capítulo 2

Usando DB Browser for SQLite y la base de datos TechStore:

1. Escribe una consulta que muestre todas las columnas de la tabla `customers` limitada a 10 filas.
2. Muestra solo los nombres (`first\_name`) de los clientes.
3. Muestra el nombre y el precio de los productos, ordenados por precio de mayor a menor, limitado a 5.
4. ¿Cuántos productos hay con stock mayor que 0? (pista: `SELECT COUNT(\*) FROM products WHERE stock > 0`)
5. Muestra los 3 productos más baratos con nombre y precio.
6. Muestra los nombres de los empleados y su puesto, ordenados por fecha de contratación.
7. Usa alias para mostrar el nombre del producto como "Producto" y el precio como "Precio".
8. ¿Qué categorías existen? Consulta la tabla `categories`.
9. Muestra los 10 pedidos más recientes con su fecha y total.
10. ¿Cuántas filas tiene la tabla `order\_items`?

Checklist de autoevaluación

- ☐ Entiendo qué es SQL y para qué sirve
- ☐ Sé escribir `SELECT` con columnas específicas
- ☐ Sé usar `LIMIT` para controlar resultados
- ☐ Sé ordenar resultados con `ORDER BY` y `DESC`/`ASC`
- ☐ Sé usar `AS` para crear alias
- ☐ Sé escribir comentarios en SQL

Capítulo 3: SQL — Filtrando con Precisión

El poder de WHERE

La cláusula `WHERE` filtra registros según una condición. Solo se devuelven las filas que cumplen la condición.

```
SELECT name, price, stock
FROM products
WHERE stock = 0;
```

Esta consulta encuentra todos los productos agotados.

Operadores de comparación

Operador	Significado	Ejemplo
<code>=</code>	Igual a	<code>WHERE stock = 0</code>
<code>!=</code> o <code><></code>	Distinto de	<code>WHERE category_id <> 3</code>
<code>></code>	Mayor que	<code>WHERE price > 500</code>
<code><</code>	Menor que	<code>WHERE price < 50</code>
<code>>=</code>	Mayor o igual	<code>WHERE price >= 1000</code>
<code><=</code>	Menor o igual	<code>WHERE stock <= 10</code>

```
-- Productos con precio mayor a 500€
SELECT name, price FROM products WHERE price > 500;

-- Productos con stock bajo (10 o menos unidades)
SELECT name, stock FROM products WHERE stock <= 10;
```

AND, OR, NOT: combinando condiciones

AND: todas las condiciones deben cumplirse

```
-- Productos caros Y con buen stock
SELECT name, price, stock
FROM products
WHERE price > 300 AND stock > 50;
```

OR: al menos una condición debe cumplirse

```
-- Productos de categoría 1 (Portátiles) o categoría 3 (Tablets)
SELECT name, category_id, price
FROM products
WHERE category_id = 1 OR category_id = 3;
```

Combinando AND y OR

Usa paréntesis para agrupar condiciones, igual que en matemáticas:

```
-- Productos de categoría 1 o 2, con precio mayor a 500
SELECT name, category_id, price
FROM products
WHERE (category_id = 1 OR category_id = 2) AND price > 500;
```

NOT: niega una condición

```
-- Productos que NO son de categoría 1
SELECT name, category_id FROM products WHERE NOT category_id = 1;
```

IN: múltiples valores posibles

`IN` es una forma más limpia de escribir múltiples condiciones `OR`:

```
-- En lugar de: category_id = 1 OR category_id = 3 OR category_id = 5
SELECT name, category_id, price
FROM products
WHERE category_id IN (1, 3, 5);
```

BETWEEN: rangos de valores

```
-- Productos entre 100 y 300 euros
SELECT name, price
FROM products
WHERE price BETWEEN 100 AND 300;
```

`BETWEEN` incluye los extremos. Es equivalente a `price >= 100 AND price <= 300`.

LIKE: búsqueda de texto parcial

```
-- Productos que contienen "Pro" en el nombre
SELECT name FROM products WHERE name LIKE '%Pro%';
```

El símbolo `%` significa "cualquier secuencia de caracteres":

Patrón	Significado
`'Pro%`	Empieza con "Pro"
`'%Pro`	Termina con "Pro"
`'%Pro%`	Contiene "Pro" en cualquier parte
`'Pro_`	"Pro" seguido de exactamente 1 carácter

NULL: valores ausentes

`NULL` significa "valor desconocido" o "sin dato". No es lo mismo que o o cadena vacía. No puedes usar `=` con NULL. Usa `IS NULL` o `IS NOT NULL`:

```
-- Productos sin proveedor asignado
SELECT name, supplier FROM products WHERE supplier IS NULL;

-- Productos con proveedor
SELECT name, supplier FROM products WHERE supplier IS NOT NULL;
```

Combinando todo

```
-- Productos de categorías 1, 2 o 3, con precio entre 200 y 1000,
-- con stock disponible, ordenados por precio descendente
SELECT
    name AS producto,
    price AS precio,
    stock AS inventario
FROM products
WHERE category_id IN (1, 2, 3)
    AND price BETWEEN 200 AND 1000
    AND stock > 0
ORDER BY price DESC;
```

Filtros en la tabla customers

```
-- Clientes de Madrid
SELECT first_name, last_name, city, region
FROM customers
WHERE city = 'Madrid';

-- Clientes de Andalucía o Cataluña
SELECT first_name, last_name, region
FROM customers
WHERE region IN ('Andalucía', 'Cataluña');

-- Clientes registrados después de 2025
SELECT first_name, last_name, registration_date
FROM customers
WHERE registration_date >= '2025-01-01';
```

Ejercicios del Capítulo 3

1. Encuentra todos los productos con precio mayor a 1000€.
2. ¿Cuántos productos tienen stock igual a 0?
3. Muestra los productos de la categoría 4 (Auriculares) con precio menor a 100€.
4. Encuentra clientes de las regiones 'Madrid' o 'Cataluña'.
5. Muestra los pedidos con total entre 100 y 500€, ordenados por total descendente.
6. ¿Cuántos pedidos están cancelados (status = 'Cancelado')?
7. Encuentra productos cuyo nombre contiene "Ultra".

8. Muestra los empleados contratados después del 1 de enero de 2024.
9. ¿Cuántos productos tienen precio NULL? (ninguno, pero practica `IS NULL`)
10. Muestra los 10 productos más caros con stock disponible (stock > 0).

Checklist de autoevaluación

- ☐ Sé usar `WHERE` con operadores de comparación
- ☐ Sé combinar condiciones con `AND`, `OR` y `NOT`
- ☐ Sé usar `IN` para múltiples valores
- ☐ Sé usar `BETWEEN` para rangos
- ☐ Sé usar `LIKE` para búsqueda por patrón
- ☐ Entiendo la diferencia entre `NULL` y otros valores
- ☐ Sé filtrar valores nulos con `IS NULL` / `IS NOT NULL`

Capítulo 4: Excel — Importar y Transformar Datos

Excel como herramienta de análisis

Excel es la herramienta más utilizada por analistas de datos en todo el mundo. Aunque no es tan potente como SQL o Python para grandes volúmenes, es perfecta para:

- Análisis rápidos y exploratorios
- Visualización de datos
- Presentación de resultados a negocio
- Limpieza y transformación de datos pequeños

Un analista competente usa Excel a diario, combinado con SQL y Python.

Importar datos a Excel

Desde CSV

1. Abre Excel
2. Ve a "Datos" > "Obtener datos" > "Desde archivo" > "Desde CSV"
3. Selecciona `datos\_ventas.csv` de la carpeta `codigos/`
4. Excel detectará automáticamente el separador y los tipos de datos
5. Haz clic en "Cargar"

Desde texto con pegado especial

Otra forma común es copiar datos de un sistema y pegarlos en Excel:

1. Copia los datos (Ctrl+C)
2. En Excel, selecciona la celda A1
3. Pega con "Pegado especial" > "Texto" o simplemente Ctrl+V

La estructura de una hoja de cálculo

Excel organiza los datos en:

- **Filas:** cada fila es un registro
- **Columnas:** cada columna es un campo
- **Celdas:** intersección de fila y columna (ej: A1, B2, C10)

En nuestro archivo `datos\_ventas.csv`:

Fila	order_id	order_date	status	total	customer_name	city	region	employee_name
1	1	2024-03-15	Completado	1250.50	Ana García	Madrid	Madrid	Carlos Ruiz
2	2	2024-03-15	Pendiente	89.99	Luis Pérez	Barcelona	Cataluña	María López

Formato básico de celdas

Números

Selecciona las celdas y usa:

- **Ctrl+Shift+1**: Número con separador de miles
- **Ctrl+Shift+4**: Moneda (€)
- **Ctrl+Shift+5**: Porcentaje
- **Ctrl+Shift+3**: Fecha

O desde la cinta: "Inicio" > "Número" > selecciona el formato.

Texto

- **Negrita**: Ctrl+N
- **Cursiva**: Ctrl+K
- **Alinear texto**: izquierda, centro, derecha
- **Ajustar texto**: "Inicio" > "Alinear" > "Ajustar texto"

Colores y bordes

- **Color de relleno**: "Inicio" > "Fuente" > cubo de pintura
- **Color de fuente**: "Inicio" > "Fuente" > A
- **Bordes**: "Inicio" > "Fuente" > bordes

Filtros y ordenación

Autofiltro

1. Selecciona cualquier celda dentro de los datos
2. Ctrl+Shift+L o "Datos" > "Filtro"
3. Aparecen flechas en cada encabezado
4. Haz clic en una flecha para filtrar por valor, color o condición

Filtro rápido: selecciona una región específica
→ Solo verás ventas de esa región

Ordenar

1. Selecciona una celda en la columna a ordenar
2. "Datos" > "AZ" (ascendente) o "ZA" (descendente)
3. O: "Datos" > "Ordenar" para múltiples niveles

Tablas de Excel (Ctrl+T)

Convertir un rango en "Tabla" da superpoderes:

1. Selecciona cualquier celda dentro de los datos
2. Ctrl+T (o "Insertar" > "Tabla")
3. Asegúrate de que "Mi tabla tiene encabezados" está marcado

Ventajas de las tablas:

- Filtros automáticos
- El formato se mantiene al añadir filas
- Las referencias se actualizan solas
- Las fórmulas se auto-rellenan

Fórmulas básicas

SUMA

```
=SUMA(D2:D101) -- Suma de la columna total (D)
```

PROMEDIO

```
=PROMEDIO(D2:D101) -- Promedio de los totales
```

CONTAR

```
=CONTARA(B2:B101) -- Cuenta celdas con texto (fechas de pedido)
=CONTAR(D2:D101) -- Cuenta celdas con números
```

CONTAR.SI

```
=CONTAR.SI(C2:C101; "Completado") -- Cuenta pedidos completados
```

SUMAR.SI

```
=SUMAR.SI(C2:C101; "Completado"; D2:D101) -- Suma total de completados
```

Análisis rápido con tus datos de ventas

Carga `datos\_ventas.csv` y practica:

1. Convierte el rango en tabla (Ctrl+T)
2. Aplica formato de moneda a la columna `total`
3. Filtra para mostrar solo pedidos "Completado"
4. Ordena por `total` descendente para ver los pedidos más grandes
5. Calcula el total de ventas en una celda: `=SUMA(tabla1[total])`
6. Calcula el promedio por pedido: `=PROMEDIO(tabla1[total])`

Ejercicios del Capítulo 4

1. Importa `datos\_ventas.csv` a Excel y conviértelo en tabla.
2. Aplica formato de moneda (€) a la columna `total`.
3. ¿Cuál es el total de ventas? Usa `=SUMA()` en una celda vacía.

4. ¿Cuántos pedidos hay? Usa `=CONTAR()` o mira el número de filas.
5. Filtra los pedidos de la región "Madrid". ¿Cuántos hay?
6. Ordena los pedidos por `total` de mayor a menor. ¿Cuál es el pedido más grande?
7. Usa `=PROMEDIO()` para calcular el ticket promedio.
8. Usa `=CONTAR.SI()` para contar cuántos pedidos están "Cancelado".
9. Usa `=SUMAR.SI()` para sumar el total de pedidos "Completado".
10. ¿Qué porcentaje de pedidos están completados?

Checklist de autoevaluación

- ☐ Sé importar datos CSV a Excel
- ☐ Sé aplicar formato a celdas (números, moneda, texto)
- ☐ Sé usar autofiltro para filtrar datos
- ☐ Sé ordenar datos por una o más columnas
- ☐ Sé crear tablas con Ctrl+T
- ☐ Sé usar fórmulas básicas: SUMA, PROMEDIO, CONTAR, CONTAR.SI

Capítulo 5: Python — Variables y Tipos de Datos

¿Por qué Python?

Python es el lenguaje más popular en análisis de datos por tres razones:

1. **Sintaxis simple:** Se lee como inglés
2. **Ecosistema increíble:** Pandas, NumPy, Matplotlib
3. **Comunidad masiva:** Siempre encuentras ayuda

No necesitas ser programador. Python para análisis de datos se aprende haciendo.

Tu primer programa

Abre VS Code, crea un archivo `hola_analista.py` y escribe:

```
print(";Hola, analista de datos!")
```

Ejecuta con: botón derecho > "Run Python" o en terminal: `python hola_analista.py`

Variables: guardar información

Una variable es como una caja donde guardas un valor:

```
nombre = "TechStore"
año_fundacion = 2018
ventas_totales = 1250000.50
tiene_tienda_online = True
```

Reglas para nombres de variables

- Usa letras, números y guiones bajos
- No empieces con número
- No uses palabras reservadas (``if``, ``for``, ``while``, etc.)
- Usa ``snake_case``: ``ventas_totales``, ``nombre_producto``

Tipos de datos básicos

```
# Texto (string)
nombre = "TechStore"
mensaje = 'Bienvenidos'
```

```
# Números enteros (int)
empleados = 25
productos = 509

# Números decimales (float)
precio = 1299.99
iva = 0.21

# Booleanos (bool)
activo = True
en_oferta = False

# Tipo de dato None (nulo)
descuento = None
```

Puedes saber el tipo de cualquier valor con ``type()`:`

```
print(type(nombre))    # <class 'str'>
print(type(empleados)) # <class 'int'>
print(type(precio))    # <class 'float'>
print(type(activo))    # <class 'bool'>
```

Conversión entre tipos

```
precio_str = "1299.99"
precio_num = float(precio_str) # Convierte texto a número
print(precio_num + 100)        # 1399.99

empleados = 25
print("Tenemos " + str(empleados) + " empleados") # Convierte número a texto
```

F-strings (la forma moderna)

```
nombre = "TechStore"
empleados = 25
print(f"{nombre} tiene {empleados} empleados")
# TechStore tiene 25 empleados
```

Operadores básicos

Aritméticos

```
a = 10
b = 3

print(a + b) # 13 (suma)
print(a - b) # 7 (resta)
print(a * b) # 30 (multiplicación)
print(a / b) # 3.33 (división real)
print(a // b) # 3 (división entera)
```

```
print(a % b)    # 1    (módulo / resto)
print(a ** b)   # 1000 (potencia)
```

Comparación

```
print(10 > 5)    # True
print(10 == 5)   # False
print(10 != 5)   # True
print(10 >= 10)  # True
```

Input del usuario

```
nombre = input("¿Cómo te llamas? ")
print(f"Bienvenido al análisis de datos, {nombre}")
```

Un ejemplo con datos de TechStore

```
# Calcular precio con IVA
precio_sin_iva = 1299.99
iva = 0.21
precio_con_iva = precio_sin_iva * (1 + iva)
print(f"Precio sin IVA: {precio_sin_iva}€")
print(f"Precio con IVA: {precio_con_iva:.2f}€")

# Calcular margen de beneficio
coste = 850.00
margen = precio_sin_iva - coste
porcentaje_margen = (margen / precio_sin_iva) * 100
print(f"Margen: {margen:.2f}€ ({porcentaje_margen:.1f}%)")

# Valor del inventario
unidades = 150
valor_inventario = precio_sin_iva * unidades
print(f"Valor inventario: {valor_inventario:,.2f}€")
```

Errores comunes (y cómo solucionarlos)

```
# Error: mezclar tipos
precio = "1299.99"
# print(precio + 100) # TypeError: can only concatenate str to str

# Solución: convertir
print(float(precio) + 100) # 1399.99

# Error: variable no definida
# print(ventas_2026) # NameError

# Solución: definir la variable primero
ventas_2026 = 1500000
```

```
print(ventas_2026)
```

Ejercicios del Capítulo 5

Escribe y ejecuta cada ejercicio en VS Code.

1. Crea una variable `nombre\_tienda` con valor "TechStore" y muéstrala.
2. Crea variables `ingresos` (1500000) y `gastos` (950000). Calcula y muestra el beneficio.
3. Pide al usuario su nombre y muestra un saludo personalizado con f-string.
4. Calcula el precio con IVA (21%) de un producto que cuesta 499.99€.
5. Convierte el string "1299.99" a float y súmalo 200. Muestra el resultado.
6. Calcula cuántos días han pasado desde el 1 de enero de 2024 hasta hoy. Usa `from datetime import date` y haz la resta.
7. Crea una variable `stock` con valor 50. Simula una venta de 3 unidades y muestra el stock restante usando f-string.
8. Calcula el porcentaje de mujeres en una empresa: de 25 empleados, 12 son mujeres.
9. Pide al usuario dos números, súmalos y muestra el resultado (recuerda convertir el input).
10. Usa `type()` para verificar los tipos de: 42, 3.14, "Hola", True, None.

Proyecto rápido: Calculadora de métricas TechStore

```
# Calculadora rápida de métricas
print("=== TechStore - Calculadora de Métricas ===")

ingresos = float(input("Ingresos totales: "))
gastos = float(input("Gastos totales: "))
clientes = int(input("Número de clientes: "))

beneficio = ingresos - gastos
margen = (beneficio / ingresos) * 100
ingreso_por_cliente = ingresos / clientes

print(f"\n=== Resumen ===")
print(f"Beneficio: {beneficio:,.2f}€")
print(f"Margen: {margen:.1f}%")
print(f"Ingreso por cliente: {ingreso_por_cliente:.2f}€")
```

Checklist de autoevaluación

- [] Sé crear y ejecutar un archivo Python
- [] Sé crear variables con nombres descriptivos
- [] Conozco los tipos básicos: int, float, str, bool, None
- [] Sé convertir entre tipos (int a str, str a float)
- [] Sé usar f-strings para formatear texto
- [] Sé usar operadores aritméticos y de comparación
- [] Sé obtener input del usuario con input()

Checkpoint 1: Analizando Ventas Básicas

Proyecto integrador del Proyecto 1

En este checkpoint vas a combinar todo lo aprendido en los capítulos 1-5 para realizar un análisis completo de las ventas de TechStore usando las tres herramientas.

Parte 1: SQL — Explora los datos

Abre DB Browser, conecta `techstore.db` y responde:

```
-- 1.1 ¿Cuántos pedidos hay en total?
SELECT COUNT(*) FROM orders;

-- 1.2 ¿Cuánto es el total de ventas?
SELECT ROUND(SUM(total), 2) FROM orders;

-- 1.3 ¿Cuál es el ticket promedio?
SELECT ROUND(AVG(total), 2) FROM orders;

-- 1.4 ¿Cuántos pedidos por estado?
SELECT status, COUNT(*) AS cantidad
FROM orders
GROUP BY status;

-- 1.5 ¿Cuál es el pedido más caro y el más barato?
SELECT MAX(total) AS maximo, MIN(total) AS minimo FROM orders;

-- 1.6 ¿Cuántos productos hay por categoría?
SELECT category_id, COUNT(*) AS cantidad
FROM products
GROUP BY category_id
ORDER BY cantidad DESC;

-- 1.7 ¿Cuáles son los 5 productos más caros?
SELECT name, price FROM products ORDER BY price DESC LIMIT 5;

-- 1.8 ¿Cuántos clientes hay por región?
SELECT region, COUNT(*) AS cantidad
FROM customers
GROUP BY region
ORDER BY cantidad DESC;
```

Parte 2: Excel — Analiza las ventas

1. Importa `datos\_ventas.csv` a Excel y conviértelo en tabla
2. Calcula:
 - Total de ventas: `=SUMA(tabla[total])`
 - Promedio por pedido: `=PROMEDIO(tabla[total])`
 - Pedido máximo: `=MAX(tabla[total])`
 - Pedido mínimo: `=MIN(tabla[total])`
 - Conteo de pedidos: `=CONTAR(tabla[total])`
3. Filtra por región "Madrid" y calcula el total de ventas de Madrid
4. Filtra por estado "Completado" y calcula el total
5. ¿Qué porcentaje representan los pedidos completados sobre el total?

Parte 3: Python — Calcula métricas

Crea un archivo ` analisis\_ventas.py`:

```
# Análisis básico de ventas TechStore

print("=== MÉTRICAS DE TECHSTORE ===\n")

# Datos del negocio
ingresos = 1250000.50
gastos = 875000.00
clientes = 250
productos = 509
empleados = 25
pedidos = 6000
cancelados = 450
devueltos = 120

# Cálculos
beneficio = ingresos - gastos
margen = (beneficio / ingresos) * 100
pedidos_completados = pedidos - cancelados - devueltos
tasa_completados = (pedidos_completados / pedidos) * 100
ingreso_por_cliente = ingresos / clientes
ingreso_por_pedido = ingresos / pedidos
productos_por_empleado = productos / empleados

# Resultados
print(f"Ingresos:           {ingresos:>12,.2f}€")
print(f"Gastos:             {gastos:>12,.2f}€")
print(f"Beneficio:            {beneficio:>12,.2f}€")
print(f"Margen:               {margen:>11.1f}%")
print(f"Clientes:             {clientes:>12}")
print(f"Pedidos totales:       {pedidos:>12}")
print(f"Pedidos completados:   {pedidos_completados:>12}")
print(f"Tasa de completados:   {tasa_completados:>11.1f}%")
print(f"Ingreso por cliente:    {ingreso_por_cliente:>12.2f}€")
```

```
print(f"Ingreso por pedido: {ingreso_por_pedido:>12.2f}€")
```

Parte 4: Preguntas de negocio

Basándote en tu análisis, responde:

1. ¿Cuántos pedidos se completan vs se cancelan? ¿Es buena la tasa?
2. ¿Cuál es el ticket promedio? ¿Te parece alto o bajo para una tienda de electrónica?
3. ¿Qué región tiene más clientes? ¿Por qué crees?
4. ¿Cuál es la categoría con más productos?
5. ¿Qué métrica crees que es más importante para el negocio?

Entregable

Guarda tu análisis en una carpeta `proyecto\_1` con:

- `consultas.sql` — Tus consultas SQL
- ` analisis\_ventas.xlsx` — Tu archivo Excel con fórmulas
- ` analisis\_ventas.py` — Tu script Python

Checklist de autoevaluación del Proyecto 1

- ☐ Sé consultar datos con SELECT, WHERE, ORDER BY, LIMIT
- ☐ Sé filtrar con operadores de comparación, IN, BETWEEN, LIKE
- ☐ Sé importar CSV a Excel y formatear datos
- ☐ Sé usar tablas de Excel (Ctrl+T) y fórmulas básicas
- ☐ Sé crear variables y usar tipos de datos en Python
- ☐ Sé usar f-strings y operadores aritméticos
- ☐ Completé el análisis integrado en las 3 herramientas
- ☐ Guardé mi proyecto en una carpeta organizada

Capítulo 6: SQL — Agrupando y Resumiendo Datos

Funciones de agregación

Las funciones de agregación resumen múltiples filas en un solo resultado. Son la base del análisis de datos en SQL.

```
SELECT COUNT(*) AS total_pedidos,  
       ROUND(SUM(total), 2) AS ingresos_totales,  
       ROUND(AVG(total), 2) AS ticket_promedio,  
       MIN(total) AS pedido_minimo,  
       MAX(total) AS pedido_maximo  
FROM orders;
```

Función	Descripción	Ejemplo
<code>COUNT(*)</code>	Cuenta filas	<code>COUNT(*) = 6000</code>
<code>COUNT(columna)</code>	Cuenta valores no NULL	<code>COUNT(supplier)</code>
<code>SUM(columna)</code>	Suma de valores	<code>SUM(total)</code>
<code>AVG(columna)</code>	Promedio	<code>AVG(total)</code>
<code>MIN(columna)</code>	Valor mínimo	<code>MIN(total)</code>
<code>MAX(columna)</code>	Valor máximo	<code>MAX(total)</code>

GROUP BY: agrupar por categorías

`GROUP BY` agrupa filas que comparten un valor. Se usa con funciones de agregación.

```
SELECT status, COUNT(*) AS cantidad, ROUND(SUM(total), 2) AS total  
FROM orders  
GROUP BY status;
```

Resultado:

status	cantidad	total
Cancelado	450	225,000
Completado	4,800	4,800,000
Envío	300	300,000
Pendiente	450	350,000

GROUP BY con múltiples columnas

```
SELECT c.region, o.status,
       COUNT(*) AS pedidos,
       ROUND(SUM(o.total), 2) AS ingresos
FROM orders o
JOIN customers c ON o.customer_id = c.id
GROUP BY c.region, o.status
ORDER BY c.region, ingresos DESC;
```

HAVING: filtrar grupos

‘WHERE’ filtra filas antes de agrupar. ‘HAVING’ filtra grupos después de agrupar.

```
SELECT customer_id,
       COUNT(*) AS pedidos,
       ROUND(SUM(total), 2) AS gasto_total
FROM orders
GROUP BY customer_id
HAVING COUNT(*) >= 10
ORDER BY gasto_total DESC;
```

Solo clientes con 10 o más pedidos aparecen en el resultado.

Orden de ejecución en SQL

Es importante entender el orden en que SQL ejecuta las cláusulas:

1. ‘FROM’ — Selecciona las tablas
2. ‘WHERE’ — Filtra filas individuales
3. ‘GROUP BY’ — Agrupa las filas
4. ‘HAVING’ — Filtra los grupos
5. ‘SELECT’ — Selecciona las columnas
6. ‘ORDER BY’ — Ordena el resultado
7. ‘LIMIT’ — Limita el número de filas

Análisis práctico con TechStore

Ventas por categoría de producto

```
SELECT c.name AS categoria,
       COUNT(DISTINCT oi.order_id) AS pedidos,
       SUM(oi.quantity) AS unidades_vendidas,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN categories c ON p.category_id = c.id
GROUP BY c.name
ORDER BY ingresos DESC;
```

Ventas mensuales

```
SELECT strftime('%Y-%m', order_date) AS mes,
       COUNT(*) AS pedidos,
```

```

        ROUND(SUM(total), 2) AS ingresos
FROM orders
WHERE status = 'Completado'
GROUP BY mes
ORDER BY mes;

```

Productos con mejor margen

```

SELECT p.name AS producto,
       p.price AS precio,
       p.cost AS coste,
       ROUND((p.price - p.cost) / p.price * 100, 1) AS margen_pct,
       SUM(oi.quantity) AS unidades_vendidas
FROM products p
JOIN order_items oi ON p.id = oi.product_id
GROUP BY p.id
HAVING unidades_vendidas > 10
ORDER BY margen_pct DESC
LIMIT 10;

```

Ejercicios del Capítulo 6

1. ¿Cuántos pedidos hay por cada estado (status)?
2. ¿Cuál es el total de ingresos por región?
3. ¿Cuántos productos hay en cada categoría? Muestra el nombre de la categoría y el conteo.
4. ¿Cuáles son las 5 ciudades con más clientes?
5. ¿Cuál es el ticket promedio por región?
6. ¿Cuántos pedidos ha hecho cada cliente? Muestra solo los que han hecho más de 5 pedidos.
7. ¿Cuál es el producto más vendido por unidades totales?
8. ¿Cuántos pedidos se hicieron cada mes en 2024?
9. ¿Cuál es el ingreso total generado por cada empleado (vendedor)?
10. ¿Qué categoría tiene el precio promedio más alto?

Checklist de autoevaluación

- ☐ Sé usar funciones de agregación: COUNT, SUM, AVG, MIN, MAX
- ☐ Sé agrupar datos con GROUP BY
- ☐ Sé filtrar grupos con HAVING
- ☐ Entiendo la diferencia entre WHERE y HAVING
- ☐ Sé usar GROUP BY con múltiples columnas
- ☐ Entiendo el orden de ejecución de SQL

Capítulo 7: SQL — Conectando Tablas con JOINS

¿Por qué necesitamos JOINS?

Los datos en una base de datos relacional están distribuidos en varias tablas. Para responder preguntas de negocio, necesitas combinarlas.

```
-- Sin JOIN: solo ves IDs, no nombres
SELECT customer_id, total FROM orders LIMIT 5;

-- Con JOIN: ves el nombre del cliente
SELECT o.id, c.first_name, c.last_name, o.total
FROM orders o
JOIN customers c ON o.customer_id = c.id
LIMIT 5;
```

INNER JOIN: solo registros que coinciden

`INNER JOIN` devuelve solo las filas donde hay correspondencia en ambas tablas.

```
SELECT o.id AS pedido,
       c.first_name || ' ' || c.last_name AS cliente,
       o.order_date, o.total
FROM orders o
INNER JOIN customers c ON o.customer_id = c.id
LIMIT 10;
```

Pedido	Cliente	Fecha	Total
1	Ana García	2024-03-15	1250.50
2	Luis Pérez	2024-03-15	89.99
...			

LEFT JOIN: todo de la izquierda, coincidencias de la derecha

`LEFT JOIN` devuelve todas las filas de la tabla izquierda. Si no hay coincidencia, las columnas de la derecha son NULL.

```
SELECT p.name AS producto, oi.order_id, oi.quantity
FROM products p
LEFT JOIN order_items oi ON p.id = oi.product_id
```

```
WHERE oi.order_id IS NULL;
```

Esto encuentra productos que nunca se han vendido.

JOIN con múltiples tablas

Los datos de TechStore requieren unir varias tablas para obtener información completa:

```
SELECT o.id AS pedido,
       c.first_name || ' ' || c.last_name AS cliente,
       c.region,
       e.first_name || ' ' || e.last_name AS vendedor,
       o.order_date, o.total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN employees e ON o.employee_id = e.id
WHERE o.status = 'Completado'
ORDER BY o.total DESC
LIMIT 10;
```

JOIN con agregación

```
SELECT c.name AS categoria,
       COUNT(DISTINCT o.id) AS pedidos,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM categories c
JOIN products p ON c.id = p.category_id
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'Completado'
GROUP BY c.name
ORDER BY ingresos DESC;
```

Alias de tablas

Usa alias cortos para simplificar consultas con muchos JOINS:

```
-- Sin alias
SELECT orders.id, customers.first_name, products.name
FROM orders
JOIN customers ON orders.customer_id = customers.id
JOIN order_items ON orders.id = order_items.order_id
JOIN products ON order_items.product_id = products.id;

-- Con alias
SELECT o.id, c.first_name, p.name
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id;
```


Uniendo una tabla consigo misma (Self JOIN)

A veces necesitas comparar filas dentro de la misma tabla:

```
-- Productos en la misma categoría con precio similar
SELECT a.name AS producto_a,
       b.name AS producto_b,
       a.category_id, a.price
FROM products a
JOIN products b ON a.category_id = b.category_id
                AND a.id < b.id
                AND ABS(a.price - b.price) < 10

LIMIT 20;
```

Ejercicios del Capítulo 7

1. Muestra los 10 pedidos más recientes con nombre del cliente y región.
2. ¿Qué empleado ha gestionado más pedidos? Muestra su nombre y conteo.
3. Lista los productos con su nombre de categoría, ordenados por precio descendente.
4. ¿Cuántos pedidos ha hecho cada cliente? Muestra nombre y conteo, ordenado de mayor a menor.
5. ¿Qué producto ha generado más ingresos? Necesitas joins con order_items y products.
6. Muestra todos los clientes que NO han hecho ningún pedido (usa LEFT JOIN y IS NULL).
7. ¿Qué vendedor ha generado más ingresos? Muestra su nombre y el total.
8. Para cada categoría, muestra el producto más caro y el más barato.
9. ¿Cuántos productos diferentes compró cada cliente? Muestra cliente y conteo de productos distintos.
10. Muestra el top 5 de productos más vendidos con su nombre de categoría y unidades vendidas.

Checklist de autoevaluación

- [] Entiendo qué son los JOINS y por qué son necesarios
- [] Sé usar INNER JOIN para combinar tablas
- [] Sé usar LEFT JOIN para incluir registros sin coincidencia
- [] Sé unir más de 2 tablas en una consulta
- [] Sé usar alias de tablas
- [] Entiendo la diferencia entre INNER y LEFT JOIN

Capítulo 8: Excel — Tablas Dinámicas

¿Qué es una tabla dinámica?

Una tabla dinámica (pivot table) es la herramienta más potente de Excel para resumir y analizar datos. Con pocos clics puedes convertir miles de filas en un informe resumido.

En SQL escribes:

```
SELECT region, COUNT(*), SUM(total)
FROM ventas GROUP BY region;
```

En Excel, una tabla dinámica hace lo mismo sin escribir código.

Crear tu primera tabla dinámica

1. Abre `datos\_ventas.csv` en Excel
2. Conviértelo en tabla (Ctrl+T)
3. Ve a "Insertar" > "Tabla dinámica"
4. Confirma el rango y elige "Nueva hoja de cálculo"
5. En el panel de campos, arrastra:

- **Filas:** `region`

- **Valores:** `total` (arrastra 3 veces)

6. En los valores, cambia la configuración:

- Primer `total`: Suma "Ingresos totales"
- Segundo `total`: Cuenta "Pedidos"
- Tercer `total`: Promedio "Ticket promedio"

Resultado: una tabla que muestra ingresos, pedidos y ticket promedio por región.

Componentes de una tabla dinámica

FILTROS	———	Filtran toda la tabla (ej: filtrar por año)
↓		
COLUMNAS	———	Valores que van en columnas (ej: años)
↓		
FILAS	———	Valores que van en filas (ej: regiones)
↓		
VALORES	———	Métricas a calcular (ventas, conteos, etc.)

Arrastrar campos: ejemplos prácticos

Ejemplo 1: Ventas por región y estado

- **Filas:** `region`
- **Columnas:** `status`
- **Valores:** Suma de `total`

Ejemplo 2: Clientes por ciudad (conteo)

- **Filas:** `city`
- **Valores:** Conteo de `customer\_name`

Ejemplo 3: Ventas por mes

- **Filas:** `order\_date` (agrupado por meses)
- **Valores:** Suma de `total`

Agrupar fechas en tablas dinámicas

Excel agrupa fechas automáticamente en años, trimestres, meses y días.

1. Arrastra `order\_date` a Filas
2. Haz clic derecho en cualquier fecha
3. Selecciona "Agrupar"
4. Elige: Años, Trimestres, Meses
5. Aceptar

Ahora tienes ventas agrupadas por año, trimestre y mes.

Segmentadores y líneas de tiempo

Segmentadores: filtros visuales

1. Selecciona la tabla dinámica
2. "Analizar tabla dinámica" > "Insertar segmentador"
3. Elige: `region`, `status`
4. Aparecen botones visuales para filtrar

Línea de tiempo: filtro temporal

1. "Analizar tabla dinámica" > "Insertar línea de tiempo"
2. Selecciona `order\_date`
3. Aparece un control deslizante de tiempo

Campos calculados

Puedes crear nuevos campos dentro de la tabla dinámica:

1. "Analizar tabla dinámica" > "Campos, elementos y conjuntos" > "Campo calculado"
2. Nombre: "Margen estimado"
3. Fórmula: `= total \* 0.3` (asumiendo 30% de margen)

Formato condicional en tablas dinámicas

1. Selecciona los valores en la tabla dinámica

2. "Inicio" > "Formato condicional"
3. Elige: "Barras de datos", "Escalas de color" o "Conjuntos de iconos"

Ejemplo: aplica barras de datos a la columna de ingresos para ver visualmente qué región vende más.

Actualizar datos fuente

Cuando los datos cambian:

1. Haz clic derecho en la tabla dinámica
2. "Actualizar"
3. O: "Datos" > "Actualizar todo"

Si añades filas al origen, expande el rango antes de actualizar.

Ejercicios del Capítulo 8

Carga `datos\_ventas.csv` y crea una tabla dinámica para cada ejercicio.

1. Crea una tabla dinámica que muestre el total de ventas por región.
2. Añade el conteo de pedidos y el ticket promedio a la tabla anterior.
3. Crea una tabla dinámica que muestre ventas por mes (agrupa fechas).
4. Usa un segmentador para filtrar por estado (status).
5. Crea una tabla con región en filas y status en columnas. ¿Qué región tiene más pedidos completados?
6. ¿Cuántos clientes hay en cada ciudad? Usa conteo de customer_name.
7. Agrupa las ventas por año. ¿Cómo ha evolucionado el negocio?
8. Aplica formato condicional con barras de datos a los ingresos por región.
9. Crea un campo calculado que muestre el IVA (21%) de cada pedido.
10. ¿Cuál es el mes con mayores ventas en 2025?

Checklist de autoevaluación

- ☐ Sé crear una tabla dinámica desde un rango de datos
- ☐ Sé arrastrar campos a filas, columnas y valores
- ☐ Sé cambiar el tipo de cálculo (suma, conteo, promedio)
- ☐ Sé agrupar fechas en tablas dinámicas
- ☐ Sé usar segmentadores y líneas de tiempo
- ☐ Sé crear campos calculados
- ☐ Sé aplicar formato condicional

Capítulo 9: Python — Estructuras de Datos y Control de Flujo

Listas: colecciones ordenadas

Una lista es una secuencia ordenada de elementos. Se crea con corchetes `[]`.

```
# Lista de productos
productos = ["Portátil Pro", "Smartphone Air", "Tablet Max", "Auriculares Ultra"]
print(productos[0])    # Portátil Pro (índice 0)
print(productos[-1])   # Auriculares Ultra (último)
print(productos[1:3])  # ["Smartphone Air", "Tablet Max"] (slicing)
```

Operaciones con listas

```
ventas_mensuales = [45000, 52000, 48000, 55000, 61000, 58000]

# Añadir elementos
ventas_mensuales.append(62000)

# Insertar en posición específica
ventas_mensuales.insert(0, 43000) # Insertar al inicio

# Eliminar elementos
ventas_mensuales.remove(48000) # Eliminar por valor
ultimo = ventas_mensuales.pop() # Eliminar y devolver el último

# Longitud
print(len(ventas_mensuales))

# Suma, máximo, mínimo
print(sum(ventas_mensuales))
print(max(ventas_mensuales))
print(min(ventas_mensuales))

# Ordenar
ventas_mensuales.sort(reverse=True) # Descendente
```

Diccionarios: pares clave-valor

Un diccionario almacena datos en pares `clave: valor`. Se crea con llaves `{}`.

```

# Diccionario de un producto
producto = {
    "nombre": "Portátil Pro",
    "precio": 1299.99,
    "stock": 50,
    "categoria": "Portátiles"
}

# Acceder a valores
print(producto["nombre"]) # Portátil Pro
print(producto.get("precio")) # 1299.99 (seguro, no da error si falta)

# Modificar valores
producto["precio"] = 1199.99
producto["stock"] -= 1 # Vender una unidad

# Añadir nuevas claves
producto["proveedor"] = "TechSupply S.L."

# Recorrer diccionario
for clave, valor in producto.items():
    print(f"{clave}: {valor}")

```

Lista de diccionarios (estructura típica de datos)

```

productos = [
    {"nombre": "Portátil Pro", "precio": 1299.99, "stock": 50},
    {"nombre": "Smartphone Air", "precio": 899.99, "stock": 120},
    {"nombre": "Tablet Max", "precio": 499.99, "stock": 80},
]

# Calcular valor total del inventario
valor_total = 0
for p in productos:
    valor_total += p["precio"] * p["stock"]
print(f"Valor inventario: {valor_total:,.2f}€")

```

Condicionales: if/elif/else

```

stock = 5

if stock == 0:
    print("Producto agotado")
elif stock <= 10:
    print(f"Stock bajo: {stock} unidades")
elif stock <= 50:
    print(f"Stock normal: {stock} unidades")
else:
    print(f"Stock alto: {stock} unidades")

```

Bucles: for y while

for: iterar sobre colecciones

```
productos = ["Portátil", "Smartphone", "Tablet"]
precios = [1299.99, 899.99, 499.99]

for i in range(len(productos)):
    print(f"{productos[i]}: {precios[i]}€")

# Más pythonico: zip
for producto, precio in zip(productos, precios):
    print(f"{producto}: {precio}€")
```

while: repetir mientras se cumpla una condición

```
stock = 50
while stock > 0:
    # Simular venta
    stock -= 1
    print(f"Vendido. Stock restante: {stock}")

print("Producto agotado")
```

Comprensión de listas (list comprehension)

Forma concisa de crear listas:

```
# Duplicar precios (versión normal)
precios = [100, 200, 300]
precios_doble = []
for p in precios:
    precios_doble.append(p * 2)

# Mismo resultado con comprensión de listas
precios_doble = [p * 2 for p in precios]

# Filtrar productos caros
precios = [100, 500, 50, 1000, 200, 800]
caros = [p for p in precios if p > 500]
print(caros) # [1000, 800]
```

Análisis práctico

```
# Analítica rápida de ventas
ventas_diarias = [1200, 3400, 2800, 4100, 3900, 4500, 3200,
                  3800, 5100, 4700, 5300, 4900, 5200, 5800]

# Métricas básicas
total = sum(ventas_diarias)
promedio = total / len(ventas_diarias)
```

```

max_venta = max(ventas_diarias)
min_venta = min(ventas_diarias)

print(f"Total: {total:,.2f}€")
print(f"Promedio diario: {promedio:,.2f}€")
print(f"Mejor día: {max_venta:,.2f}€")
print(f"Peor día: {min_venta:,.2f}€")

# Días por encima del promedio
dias_sobre_promedio = [v for v in ventas_diarias if v > promedio]
print(f"Días sobre promedio: {len(dias_sobre_promedio)} de {len(ventas_diarias)}")

# Crecimiento vs día anterior
for i in range(1, len(ventas_diarias)):
    cambio = ventas_diarias[i] - ventas_diarias[i-1]
    signo = "+" if cambio > 0 else ""
    print(f"Día {i+1}: {signo}{cambio:,.2f}€")

```

Ejercicios del Capítulo 9

1. Crea una lista con los nombres de 5 productos de TechStore.
2. Añade 3 productos más a la lista con ``append()``.
3. Crea un diccionario para un cliente con: nombre, email, ciudad, gasto_total.
4. Actualiza el gasto_total del cliente sumando 150€.
5. Dada una lista de precios ``[299, 599, 899, 1299, 1999]``, crea una nueva lista con los precios con IVA (x1.21).
6. Usa un bucle for para calcular el total de una cesta de la compra: ``[("Portátil", 1299.99), ("Ratón", 29.99), ("Teclado", 89.99)]``.
7. Filtra una lista de precios para mostrar solo los mayores de 500€.
8. Simula un inventario: crea un diccionario con 5 productos y sus stocks. Usa un bucle para vender 1 unidad de cada uno.
9. Encuentra el producto más caro en una lista de diccionarios de productos.
10. Crea una función que reciba una lista de ventas diarias y devuelva el promedio, máximo y mínimo.

Checklist de autoevaluación

- ☐ Sé crear y manipular listas
- ☐ Sé crear y manipular diccionarios
- ☐ Sé usar condicionales if/elif/else
- ☐ Sé usar bucles for para iterar colecciones
- ☐ Sé usar while para repeticiones condicionales
- ☐ Sé usar comprensión de listas
- ☐ Entiendo la estructura lista-de-diccionarios

Capítulo 10: Excel — Visualización con Gráficos

Por qué visualizar datos

Un gráfico bien diseñado comunica en segundos lo que una tabla tarda minutos en explicar. Como analista, tu trabajo no termina con los números; terminas cuando comunicas el mensaje.

Gráfico de columnas: comparar categorías

Ideal para comparar valores entre categorías.

1. Crea una tabla dinámica con ventas por región
2. Selecciona los datos
3. "Insertar" > "Gráfico de columnas"
4. Elige "Columna agrupada 2D"

Tip: ordena los datos de mayor a menor antes de graficar

Personalizar el gráfico

1. **Título:** haz doble clic y escribe "Ventas por Región 2025"
2. **Colores:** "Formato" > "Relleno de forma" > elige un color corporativo
3. **Etiquetas de datos:** haz clic derecho > "Agregar etiquetas de datos"
4. **Eje vertical:** ajusta el mínimo si es necesario

Gráfico de líneas: mostrar tendencias

Perfecto para evolución temporal.

1. Crea una tabla dinámica con ventas por mes
2. "Insertar" > "Gráfico de líneas"
3. Elige "Línea 2D con marcadores"

Línea con marcadores: muestra puntos de datos + línea de tendencia

Añadir línea de tendencia

1. Haz clic derecho en la línea
2. "Agregar línea de tendencia"
3. Elige "Lineal" o "Media móvil"

4. Muestra la ecuación R^2 si quieres

Gráfico circular: mostrar proporciones

Usa con moderación. Solo para mostrar partes de un todo (máximo 5-6 categorías).

1. Crea una tabla dinámica con ventas por categoría
2. "Insertar" > "Gráfico circular"
3. Elige "Circular 2D"

Tip: usa "Explosión" para destacar una categoría

Gráfico de barras: comparar horizontalmente

Cuando las etiquetas son largas, las barras horizontales son más legibles.

1. Crea una tabla dinámica con productos y ventas
2. "Insertar" > "Gráfico de barras"
3. Ordena de mayor a menor para mejor legibilidad

Combinar gráficos: dos ejes

Cuando tienes dos métricas con escalas diferentes (ej: ingresos en € y número de pedidos):

1. Selecciona los datos (ingresos y pedidos por mes)
2. "Insertar" > "Gráfico de columnas"
3. Haz clic derecho en la serie de pedidos > "Cambiar tipo de gráfico" > "Línea"
4. Marca "Eje secundario" para la línea

Gráficos desde una tabla dinámica

Los gráficos dinámicos se actualizan automáticamente cuando filtras:

1. Selecciona cualquier celda de la tabla dinámica
2. "Insertar" > "Gráfico dinámico"
3. Los segmentadores afectan tanto a la tabla como al gráfico

Formato profesional de gráficos

Colores corporativos

Azul oscuro: #1F4E79
Azul medio: #2E75B6
Azul claro: #9DC3E6
Naranja: #ED7D31
Gris: #A5A5A5

Consejos de diseño

1. **Elimina el ruido visual:** quita líneas de cuadrícula si no son necesarias
2. **Usa etiquetas directas:** mejor que leyendas si hay pocos datos
3. **Consistencia de color:** mismo color = misma categoría en todos los gráficos
4. **Título descriptivo:** "Ventas por Región (2025)" no "Gráfico 1"

5. **Fuente legible:** mantén el tamaño mínimo en 10pt

Dashboard en una hoja

Combina varios gráficos en una sola hoja para crear un dashboard:

1. Crea 3-4 tablas dinámicas en hojas separadas
2. Crea gráficos para cada una
3. Copia los gráficos a una hoja nueva llamada "Dashboard"
4. Conecta todos a los mismos segmentadores
5. Organiza visualmente: KPIs arriba, gráficos principales al centro

Ejercicios del Capítulo 10

Usa `datos\_ventas.csv` para todos los ejercicios.

1. Crea un gráfico de columnas con ventas totales por región.
2. Crea un gráfico de líneas con la evolución mensual de ventas (agrupa por fecha).
3. Crea un gráfico circular con la distribución de pedidos por estado.
4. Crea un gráfico de barras con las 10 ciudades con más ventas.
5. Combina ingresos y número de pedidos en un gráfico de doble eje.
6. Añade una línea de tendencia al gráfico de ventas mensuales.
7. Personaliza los colores de un gráfico con la paleta corporativa sugerida.
8. Crea un gráfico dinámico conectado a una tabla dinámica con segmentador de región.
9. Diseña un dashboard en una hoja con: ingresos por región (columnas), tendencia mensual (líneas), y distribución por estado (circular).
10. Exporta un gráfico como imagen (clic derecho > "Guardar como imagen").

Checklist de autoevaluación

- ☐ Sé crear gráficos de columnas, líneas, circulares y barras
- ☐ Sé personalizar títulos, colores y etiquetas
- ☐ Sé añadir líneas de tendencia
- ☐ Sé crear gráficos de doble eje
- ☐ Sé crear gráficos dinámicos conectados a tablas dinámicas
- ☐ Sé diseñar un dashboard básico con múltiples gráficos
- ☐ Aplico principios de diseño profesional a mis gráficos

Checkpoint 2: Top 10 Productos Más Vendidos

Proyecto integrador del Proyecto 2

En este checkpoint vas a combinar SQL, Excel y Python para identificar los productos más vendidos de TechStore y presentar un análisis completo.

Parte 1: SQL — Identifica los top productos

```
-- 2.1 Top 10 productos más vendidos por unidades
SELECT p.name AS producto,
       c.name AS categoria,
       SUM(oi.quantity) AS unidades_vendidas,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos,
       ROUND(AVG(oi.unit_price), 2) AS precio_promedio
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN categories c ON p.category_id = c.id
GROUP BY p.id
ORDER BY unidades_vendidas DESC
LIMIT 10;

-- 2.2 Top 10 productos por ingresos generados
SELECT p.name AS producto,
       c.name AS categoria,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos,
       SUM(oi.quantity) AS unidades_vendidas
FROM order_items oi
JOIN products p ON oi.product_id = p.id
JOIN categories c ON p.category_id = c.id
GROUP BY p.id
ORDER BY ingresos DESC
LIMIT 10;

-- 2.3 Ventas por categoría
SELECT c.name AS categoria,
       COUNT(DISTINCT oi.order_id) AS pedidos,
       SUM(oi.quantity) AS unidades,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM categories c
```

```

JOIN products p ON c.id = p.category_id
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'Completado'
GROUP BY c.name
ORDER BY ingresos DESC;

-- 2.4 ¿Qué cliente ha gastado más?
SELECT c.first_name || ' ' || c.last_name AS cliente,
       c.region,
       COUNT(o.id) AS pedidos,
       ROUND(SUM(o.total), 2) AS gasto_total
FROM customers c
JOIN orders o ON c.id = o.customer_id
WHERE o.status = 'Completado'
GROUP BY c.id
ORDER BY gasto_total DESC
LIMIT 10;

-- 2.5 Ventas por región y categoría (cubo de datos)
SELECT c.region,
       cat.name AS categoria,
       COUNT(DISTINCT o.id) AS pedidos,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
JOIN categories cat ON p.category_id = cat.id
WHERE o.status = 'Completado'
GROUP BY c.region, cat.name
ORDER BY c.region, ingresos DESC;

```

Parte 2: Excel — Visualiza los resultados

1. Exporta los resultados de la consulta 2.1 a CSV y cárgalos en Excel
2. Crea un gráfico de barras horizontales con el Top 10 productos
3. Crea una tabla dinámica con:
 - Filas: región
 - Columnas: categoría
 - Valores: ingresos
4. Crea un gráfico dinámico de columnas agrupadas
5. Aplica formato profesional:
 - Título descriptivo
 - Etiquetas de datos
 - Colores corporativos
 - Sin líneas de cuadrícula innecesarias

Parte 3: Python — Análisis automatizado

```
# Análisis de productos más vendidos
productos = [
    {"nombre": "Auriculares Ultra", "categoria": "Auriculares", "unidades": 245, "ingresos": 12250.00},
    {"nombre": "Smartphone Air", "categoria": "Smartphones", "unidades": 198, "ingresos": 178200.00},
    {"nombre": "Tablet Max", "categoria": "Tablets", "unidades": 167, "ingresos": 83500.00},
    {"nombre": "Portátil Pro", "categoria": "Portátiles", "unidades": 145, "ingresos": 188500.00},
    {"nombre": "Ratón Elite", "categoria": "Ratones", "unidades": 134, "ingresos": 5360.00},
    {"nombre": "Teclado Mecánico", "categoria": "Teclados", "unidades": 122, "ingresos": 10980.00},
    {"nombre": "Monitor UltraWide", "categoria": "Monitores", "unidades": 98, "ingresos": 49000.00},
    {"nombre": "Altavoz Portátil", "categoria": "Altavoces", "unidades": 87, "ingresos": 13050.00},
    {"nombre": "Disco SSD 1TB", "categoria": "Almacenamiento", "unidades": 82, "ingresos": 6560.00},
    {"nombre": "Cámara Web HD", "categoria": "Cámaras", "unidades": 76, "ingresos": 4560.00},
]

print("=== TOP 10 PRODUCTOS MÁS VENDIDOS ===\n")

# Métricas generales
total_unidades = sum(p["unidades"] for p in productos)
total_ingresos = sum(p["ingresos"] for p in productos)

print(f"Total unidades vendidas (top 10): {total_unidades}")
print(f"Total ingresos (top 10): {total_ingresos:,.2f}€\n")

# Análisis por producto
print(f"{'Producto':<25} {'Categoría':<15} {'Unidades':>10} {'Ingresos':>12} {'% Ingresos':>10}")
print("=" * 72)

for p in sorted(productos, key=lambda x: x["ingresos"], reverse=True):
    pct = (p["ingresos"] / total_ingresos) * 100
    print(f"{p['nombre']:<25} {p['categoria']:<15} {p['unidades']:>10} {p['ingresos']:>10,.2f}€ {pct:>8.1f}%")

# Concentración de ingresos
top_3_ingresos = sum(p["ingresos"] for p in sorted(productos, key=lambda x: x["ingresos"], reverse=True)[:3])
print(f"\nEl Top 3 productos concentra el {(top_3_ingresos/total_ingresos)*100:.1f}% de los ingresos")

# Recomendación
print("\n=== RECOMENDACIONES ===")
print("1. Los auriculares lideran en unidades pero no en ingresos")
print("2. Portátiles y smartphones generan más ingresos por unidad")
print("3. Considerar promociones cruzadas: portátil + ratón + teclado")
print("4. Evaluar si los productos baratos están canibalizando ventas de gama alta")
```

Parte 4: Preguntas de negocio

1. ¿Hay diferencia entre los top productos por unidades y por ingresos?
2. ¿Qué categoría genera más ingresos totales?
3. ¿Qué región tiene el ticket promedio más alto?
4. ¿Hay algún producto que se venda mucho pero genere poco margen?
5. ¿Qué recomendarías al equipo de producto basado en este análisis?

Entregable

Guarda tu análisis en `proyecto\_2/`:

- `top\_productos.sql` — Consultas SQL
- `analisis\_ventas.xlsx` — Tablas dinámicas + gráficos
- `top\_productos.py` — Análisis en Python
- `dashboard\_proyecto\_2.png` — Captura de tu dashboard

Checklist de autoevaluación del Proyecto 2

- ☐ Sé usar funciones de agregación con GROUP BY
- ☐ Sé combinar múltiples tablas con JOINS
- ☐ Sé crear tablas dinámicas en Excel
- ☐ Sé crear gráficos profesionales
- ☐ Sé usar listas, diccionarios y bucles en Python
- ☐ Sé diseñar un dashboard básico
- ☐ Completé el análisis integrado en las 3 herramientas

Capítulo 11: SQL — Subconsultas y CTEs

Subconsultas: consultas dentro de consultas

Una subconsulta es una consulta SQL dentro de otra consulta. Se usa para calcular valores intermedios.

Subconsulta en WHERE

```
-- Productos con precio superior al promedio
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

La subconsulta `(SELECT AVG(price) FROM products)` se ejecuta primero y devuelve un solo valor.

Subconsulta con IN

```
-- Clientes que han comprado productos de la categoría "Portátiles"
SELECT first_name, last_name, email
FROM customers
WHERE id IN (
    SELECT DISTINCT o.customer_id
    FROM orders o
    JOIN order_items oi ON o.id = oi.order_id
    JOIN products p ON oi.product_id = p.id
    WHERE p.category_id = 1
);
```

Subconsulta en SELECT

```
-- Porcentaje que representa cada categoría sobre el total
SELECT c.name AS categoria,
       ROUND(AVG(p.price), 2) AS precio_promedio,
       ROUND((SELECT AVG(price) FROM products), 2) AS precio_global,
       ROUND(AVG(p.price) / (SELECT AVG(price) FROM products) * 100, 1) AS porcentaje
FROM categories c
JOIN products p ON c.id = p.category_id
GROUP BY c.name
ORDER BY porcentaje DESC;
```


Subconsulta correlacionada

Una subconsulta correlacionada referencia columnas de la consulta externa.

```
-- Productos cuyo precio es mayor que el promedio de su categoría
SELECT p.name, p.price, p.category_id,
       (SELECT ROUND(AVG(p2.price), 2)
        FROM products p2
        WHERE p2.category_id = p.category_id) AS promedio_categoria
FROM products p
WHERE p.price > (SELECT AVG(p2.price)
                 FROM products p2
                 WHERE p2.category_id = p.category_id)
LIMIT 20;
```

CTEs (Common Table Expressions) con WITH

Una CTE es como una vista temporal que existe solo durante tu consulta. Hace el SQL más legible.

```
-- CTE simple
WITH ventas_por_categoria AS (
  SELECT c.name AS categoria,
         SUM(oi.quantity * oi.unit_price) AS ingresos
  FROM categories c
  JOIN products p ON c.id = p.category_id
  JOIN order_items oi ON p.id = oi.product_id
  GROUP BY c.name
)
SELECT categoria, ROUND(ingresos, 2) AS ingresos
FROM ventas_por_categoria
ORDER BY ingresos DESC;
```

Múltiples CTEs

```
WITH
clientes_top AS (
  SELECT customer_id, COUNT(*) AS pedidos, SUM(total) AS gasto
  FROM orders
  GROUP BY customer_id
  HAVING COUNT(*) > 5
),
ventas_region AS (
  SELECT c.region, SUM(o.total) AS ingresos
  FROM orders o
  JOIN customers c ON o.customer_id = c.id
  WHERE o.customer_id IN (SELECT customer_id FROM clientes_top)
  GROUP BY c.region
)
SELECT region, ROUND(ingresos, 2) AS ingresos
FROM ventas_region
ORDER BY ingresos DESC;
```

CTE recursiva

Útil para datos jerárquicos como categorías o fechas:

```
-- Generar una serie de meses
WITH RECURSIVE meses AS (
  SELECT DATE('2024-01-01') AS mes
  UNION ALL
  SELECT DATE(mes, '+1 month')
  FROM meses
  WHERE mes < DATE('2025-12-01')
)
SELECT strftime('%Y-%m', mes) AS mes
FROM meses;
```

Subconsulta vs CTE vs JOIN

Situación	Recomendación
Valor único para comparación	Subconsulta en WHERE
Necesitas reutilizar la misma subconsulta	CTE
Relacionar dos tablas	JOIN
Consulta anidada compleja	CTE (más legible)
Filtrar por agregación	HAVING o subconsulta en WHERE

Análisis práctico

```
-- Clientes con gasto superior al promedio de su región
WITH gasto_region AS (
  SELECT c.region, AVG(o.total) AS avg_gasto_region
  FROM customers c
  JOIN orders o ON c.id = o.customer_id
  GROUP BY c.region
)
SELECT c.first_name || ' ' || c.last_name AS cliente,
       c.region,
       ROUND(SUM(o.total), 2) AS gasto_total,
       ROUND(g.avg_gasto_region, 2) AS promedio_region
FROM customers c
JOIN orders o ON c.id = o.customer_id
JOIN gasto_region g ON c.region = g.region
GROUP BY c.id
HAVING SUM(o.total) > g.avg_gasto_region
ORDER BY gasto_total DESC;
```

Ejercicios del Capítulo 11

1. Encuentra los productos con precio superior al promedio general.
2. ¿Qué clientes han gastado más que el promedio de todos los clientes?
3. Usa una subconsulta en SELECT para mostrar cada pedido y el % que representa sobre el total.

4. Crea una CTE que calcule las ventas por mes, y luego consulta los meses con ventas superiores a 100,000€.
5. Usa múltiples CTEs para: (1) calcular ventas por categoría, (2) calcular ventas totales, (3) mostrar el % de cada categoría.
6. Encuentra empleados que han gestionado más pedidos que el promedio.
7. Usa una subconsulta correlacionada para mostrar productos cuyo stock está por debajo del promedio de su categoría.
8. Crea una CTE para encontrar el producto más vendido de cada categoría.
9. ¿Qué productos se han vendido más en 2025 que en 2024? (Usa dos CTEs y compáralas).
10. Encuentra clientes que han pedido productos de todas las categorías (usa subconsultas con NOT EXISTS).

Checklist de autoevaluación

- [] Sé escribir subconsultas en WHERE, SELECT y FROM
- [] Entiendo las subconsultas correlacionadas
- [] Sé crear CTEs con WITH
- [] Sé usar múltiples CTEs en una consulta
- [] Sé cuándo usar subconsulta vs CTE vs JOIN
- [] Entiendo las ventajas de legibilidad de las CTEs

Capítulo 12: SQL — Funciones de Ventana (Window Functions)

¿Qué son las funciones de ventana?

Las funciones de ventana realizan cálculos a través de un conjunto de filas relacionadas con la fila actual. A diferencia de GROUP BY, no colapsan las filas: cada fila conserva su identidad.

```
-- GROUP BY: colapsa (1 fila por grupo)
SELECT category_id, AVG(price) FROM products GROUP BY category_id;

-- Window function: no colapsa (cada fila conserva su precio + el promedio)
SELECT name, price, category_id,
       AVG(price) OVER (PARTITION BY category_id) AS promedio_categoria
FROM products
LIMIT 10;
```

Sintaxis básica

```
funcion() OVER (
  PARTITION BY columna  -- grupos (opcional)
  ORDER BY columna      -- orden dentro del grupo (opcional)
  frame_specification   -- ventana móvil (opcional)
)
```

ROW_NUMBER, RANK, DENSE_RANK

ROW_NUMBER: número único de fila

```
SELECT name, price, category_id,
       ROW_NUMBER() OVER (ORDER BY price DESC) AS posicion_global
FROM products
LIMIT 10;
```

Particionado: reiniciar el contador

```
SELECT name, price, category_id,
       ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY price DESC) AS pos_en_categoria
FROM products
```

```
LIMIT 20;
```

RANK vs DENSE_RANK

```
SELECT name, price,
       RANK() OVER (ORDER BY price DESC) AS rank,
       DENSE_RANK() OVER (ORDER BY price DESC) AS dense_rank
FROM products
WHERE price > 1500
ORDER BY price DESC;
```

name	price	rank	dense_rank
Producto A	1999.99	1	1
Producto B	1999.99	1	1
Producto C	1899.99	3	2
Producto D	1799.99	4	3

Diferencia: RANK salta números en empates, DENSE_RANK no.

Funciones de agregación como ventanas

Puedes usar SUM, AVG, COUNT como ventanas:

```
SELECT o.id AS pedido,
       o.order_date,
       o.total,
       SUM(o.total) OVER (ORDER BY o.order_date) AS acumulado,
       AVG(o.total) OVER (ORDER BY o.order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS media_movil_7d
FROM orders o
WHERE o.status = 'Completado'
ORDER BY o.order_date
LIMIT 20;
```

NTILE: dividir en grupos

```
-- Dividir productos en 4 cuartiles por precio
SELECT name, price,
       NTILE(4) OVER (ORDER BY price DESC) AS cuartil
FROM products
LIMIT 20;
```

LAG y LEAD: acceder a filas anteriores/siguientes

```
-- Comparar ventas con el mes anterior
WITH ventas_mensuales AS (
  SELECT strftime('%Y-%m', order_date) AS mes,
         ROUND(SUM(total), 2) AS ingresos
  FROM orders
  WHERE status = 'Completado'
  GROUP BY mes
```

```

)
SELECT mes, ingresos,
       LAG(ingresos, 1) OVER (ORDER BY mes) AS mes_anterior,
       ROUND(ingresos - LAG(ingresos, 1) OVER (ORDER BY mes), 2) AS variacion,
       ROUND((ingresos - LAG(ingresos, 1) OVER (ORDER BY mes)) / LAG(ingresos, 1) OVER (ORDER BY mes) * 100, 2) AS porcentaje_variacion
FROM ventas_mensuales
ORDER BY mes;

```

Frame specification (ventana móvil)

Controla exactamente qué filas incluir:

```

-- ROWS BETWEEN inicio AND fin
-- PRECEDING: filas anteriores
-- FOLLOWING: filas posteriores
-- CURRENT ROW: fila actual
-- UNBOUNDED PRECEDING: desde el inicio del grupo
-- UNBOUNDED FOLLOWING: hasta el final del grupo

-- Media móvil de 3 días
AVG(total) OVER (ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)

-- Total acumulado desde el inicio
SUM(total) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

-- Total del grupo completo (todas las filas)
SUM(total) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)

```

FIRST_VALUE y LAST_VALUE

```

-- Comparar cada producto con el más caro de su categoría
SELECT p.name, p.price, p.category_id,
       FIRST_VALUE(p.name) OVER (PARTITION BY p.category_id ORDER BY p.price DESC) AS mas_caro_categoria,
       FIRST_VALUE(p.price) OVER (PARTITION BY p.category_id ORDER BY p.price DESC) AS precio_mas_caro,
       ROUND(p.price / FIRST_VALUE(p.price) OVER (PARTITION BY p.category_id ORDER BY p.price DESC) * 100, 2) AS porcentaje_variacion
FROM products p
LIMIT 20;

```

Análisis práctico: ranking de productos

```

-- Top 3 productos más vendidos por categoría
WITH ranking AS (
    SELECT c.name AS categoria,
           p.name AS producto,
           SUM(oi.quantity) AS unidades,
           ROW_NUMBER() OVER (PARTITION BY c.id ORDER BY SUM(oi.quantity) DESC) AS posicion
    FROM categories c
    JOIN products p ON c.id = p.category_id
    JOIN order_items oi ON p.id = oi.product_id
    GROUP BY c.id, p.id
)

```

```
)  
SELECT categoria, producto, unidades  
FROM ranking  
WHERE posicion <= 3  
ORDER BY categoria, posicion;
```

Ejercicios del Capítulo 12

1. Asigna un número de fila único a todos los productos ordenados por precio descendente.
2. Divide los clientes en 5 quintiles según su gasto total.
3. Calcula el total acumulado de ventas por mes (acumulado año corrido).
4. Para cada producto, muestra cuánto representa su precio respecto al más caro de su categoría.
5. Calcula la diferencia de ventas entre cada mes y el mes anterior usando LAG.
6. Encuentra el top 3 de productos más vendidos en cada categoría (usa ROW_NUMBER con PARTITION BY).
7. Calcula una media móvil de 30 días para las ventas diarias.
8. Usa LEAD para mostrar las ventas del próximo mes junto a las del mes actual.
9. Para cada pedido, muestra qué porcentaje representa respecto al total del cliente.
10. Encuentra productos cuyo precio está por encima del promedio de su categoría (usa AVG como ventana).

Checklist de autoevaluación

- [] Entiendo qué son las funciones de ventana y cómo se diferencian de GROUP BY
- [] Sé usar ROW_NUMBER, RANK y DENSE_RANK
- [] Sé usar PARTITION BY para agrupar cálculos
- [] Sé usar funciones de agregación como ventanas
- [] Sé usar LAG y LEAD para acceder a filas adyacentes
- [] Sé usar frames (ROWS BETWEEN) para ventanas móviles
- [] Sé usar NTILE para dividir en grupos percentiles

Capítulo 13: Excel — Power Query

¿Qué es Power Query?

Power Query es la herramienta de transformación de datos de Excel. Te permite conectar, combinar, limpiar y transformar datos de múltiples orígenes sin escribir código.

Los analistas pasan el 80% de su tiempo limpiando datos. Power Query reduce drásticamente ese tiempo.

Acceder a Power Query

- Excel: "Datos" > "Obtener y transformar datos" > "Power Query Editor"
- Atajo: Alt + A + P + Q

Conectar datos desde CSV

1. "Datos" > "Desde archivo" > "Desde CSV"
2. Selecciona `datos\_detalle\_ventas.csv`
3. Se abre Power Query Editor con una vista previa

Transformaciones básicas

Promover encabezados

Cuando los encabezados están en la primera fila:

"Inicio" > "Usar la primera fila como encabezados"

Cambiar tipo de datos

Power Query detecta tipos automáticamente, pero a veces falla:

- Haz clic en el icono junto al nombre de la columna
- Selecciona el tipo correcto: texto, número entero, decimal, fecha

Eliminar filas y columnas

- Selecciona columnas > "Inicio" > "Quitar columnas"
- "Inicio" > "Quitar filas" > "Quitar filas superiores" / "Quitar filas en blanco"

Reemplazar valores

- Selecciona una columna
- "Transformar" > "Reemplazar valores"

- Busca "Madrid" y reemplaza con "Madrid (Capital)"

Dividir columnas

- Selecciona la columna a dividir
- "Transformar" > "Dividir columna"
- Opciones: por delimitador, por número de caracteres, por mayúsculas/minúsculas

Ejemplo: dividir `customer\_name` en nombre y apellido.

Combinar consultas (Merge)

Equivalente a los JOINS de SQL:

1. "Inicio" > "Combinar" > "Combinar consultas"
2. Selecciona las dos tablas
3. Elige las columnas clave
4. Selecciona el tipo de combinación:
 - Combinación interna (INNER JOIN)
 - Combinación externa izquierda (LEFT JOIN)
 - Combinación externa completa (FULL JOIN)

Ejemplo: combinar ventas con productos

1. Carga `datos\_ventas.csv` y `datos\_productos.csv`
2. En la tabla ventas, "Combinar consultas"
3. Selecciona `productos` como segunda tabla
4. Empareja `product\_id` con `id`
5. Expande la columna para incluir `product\_name` y `category`

Agregar columnas personalizadas

"Agregar columna" > "Columna personalizada"

```
= [quantity] * [unit_price]
```

Esto crea una columna `line\_total` calculada.

También puedes crear columnas condicionales:

```
if [quantity] > 2 then "Alta" else if [quantity] > 1 then "Media" else "Baja"
```

Agrupar y agregar

Equivalente a GROUP BY de SQL:

1. "Transformar" > "Agrupar por"
2. Agrupa por: `region`
3. Nueva columna: "Ingresos" con operación Suma sobre `total`

Power Query vs Excel tradicional

Operación	Excel tradicional	Power Query
Importar CSV	Archivo > Abrir	Datos > Desde CSV

Filtrar filas	Autofiltro	Quitar filas / Filtro
Combinar tablas	BUSCARV	Combinar consultas
Agrupar datos	Tabla dinámica	Agrupar por
Transformar datos	Fórmulas manuales	Editor visual
Repetir proceso	Manual cada vez	Actualizar consulta

Actualizar consultas

La gran ventaja de Power Query: cuando los datos fuente cambian:

1. "Datos" > "Actualizar todo"
2. Power Query re-ejecuta todas las transformaciones
3. El resultado se actualiza automáticamente

M: el lenguaje de Power Query

Power Query tiene su propio lenguaje llamado M. Cada operación que haces visualmente genera código M:

```
let
    Origen = Csv.Document(File.Contents("datos.csv"),...),
    #"Encabezados promovidos" = Table.PromoteHeaders(Origen, [PromoteAllScalars=true]),
    #"Tipo cambiado" = Table.TransformColumnTypes(#"Encabezados promovidos",...)
in
    #"Tipo cambiado"
```

No necesitas aprender M para usar Power Query, pero entenderlo te da superpoderes.

Ejercicios del Capítulo 13

1. Carga `datos\_detalle\_ventas.csv` en Power Query.
2. Promueve la primera fila como encabezados si no lo está.
3. Cambia el tipo de `order\_date` a fecha y `total` a decimal.
4. Filtra solo los pedidos con status "Completado".
5. Divide `customer\_name` en nombre y apellido.
6. Carga `datos\_productos.csv` y combínala con la tabla de ventas por `product\_id`.
7. Agrupa por región y calcula el total de ingresos.
8. Añade una columna personalizada que calcule el margen estimado (ingresos * 0.3).
9. Una vez terminado, carga el resultado a Excel.
10. Modifica el archivo CSV original, añade 3 filas, y actualiza la consulta. ¿Se actualiza todo automáticamente?

Checklist de autoevaluación

- ☐ Sé acceder a Power Query Editor
- ☐ Sé importar datos desde CSV
- ☐ Sé realizar transformaciones básicas (cambiar tipo, reemplazar, dividir)
- ☐ Sé combinar consultas (equivalente a JOIN)
- ☐ Sé agregar columnas personalizadas

- [] Sé agrupar y agregar datos
- [] Sé actualizar consultas cuando los datos cambian
- [] Entiendo el valor de Power Query vs Excel tradicional

Capítulo 14: Python — Funciones y Módulos

Funciones: reutilizar código

Una función es un bloque de código reutilizable. En lugar de escribir lo mismo varias veces, defines la función una vez y la llamas cuando la necesitas.

```
def calcular_ingresos(precio, cantidad):  
    return precio * cantidad  
  
# Usar la función  
total = calcular_ingresos(1299.99, 3)  
print(f"Ingresos: {total:.2f}€")
```

Parámetros y argumentos

```
def calcular_margen(precio_venta, precio_coste, iva=0.21):  
    """  
    Calcula el margen de beneficio de un producto.  
  
    Args:  
        precio_venta: Precio de venta sin IVA  
        precio_coste: Coste del producto  
        iva: Tasa de IVA (por defecto 21%)  
  
    Returns:  
        Diccionario con márgenes  
    """  
    precio_con_iva = precio_venta * (1 + iva)  
    margen_bruto = precio_venta - precio_coste  
    margen_porcentaje = (margen_bruto / precio_venta) * 100  
  
    return {  
        "precio_sin_iva": precio_venta,  
        "precio_con_iva": round(precio_con_iva, 2),  
        "coste": precio_coste,  
        "margen_bruto": round(margen_bruto, 2),  
        "margen_pct": round(margen_porcentaje, 1)  
    }
```

```
# Llamar con argumentos posicionales
resultado = calcular_margen(1299.99, 850.00)
print(resultado)

# Llamar con argumentos nombrados (más legible)
resultado = calcular_margen(precio_venta=1299.99, precio_coste=850.00, iva=0.10)
```

Parámetros por defecto y return múltiple

```
def analizar_ventas(ingresos, gastos, impuestos=0.25):
    beneficio = ingresos - gastos
    beneficio_netto = beneficio * (1 - impuestos)
    margen = (beneficio / ingresos) * 100

    return beneficio, beneficio_netto, margen

b, bn, m = analizar_ventas(1250000, 875000)
print(f"Beneficio: {b:,.2f}€")
print(f"Beneficio netto: {bn:,.2f}€")
print(f"Margen: {m:.1f}%")
```

Módulos: organizar el código

Un módulo es un archivo `.py` que contiene funciones y variables. Puedes importarlo desde otros archivos.

Importar módulos

```
# Importar un módulo completo
import math
print(math.sqrt(144)) # 12.0

# Importar funciones específicas
from math import sqrt, pi
print(sqrt(144)) # 12.0
print(pi) # 3.14159...

# Importar con alias
import math as m
print(m.sqrt(144))
```

Módulos útiles para análisis de datos

```
import math      # Operaciones matemáticas
import random    # Números aleatorios
import datetime  # Fechas y horas
import csv       # Leer/escribir CSV
import json      # Leer/escribir JSON
import os        # Operaciones del sistema
```

Trabajar con fechas (datetime)

```

from datetime import datetime, date, timedelta

# Fecha actual
hoy = date.today()
print(f"Hoy: {hoy}")

# Fecha específica
inicio = date(2024, 1, 1)
print(f"Inicio: {inicio}")

# Diferencia entre fechas
dias_transcurridos = (hoy - inicio).days
print(f"Días desde inicio: {dias_transcurridos}")

# Sumar días
dentro_de_30 = hoy + timedelta(days=30)
print(f"Dentro de 30 días: {dentro_de_30}")

# Parsear strings a fechas
fecha_str = "2024-03-15"
fecha = datetime.strptime(fecha_str, "%Y-%m-%d").date()
print(f"Parseada: {fecha}")

# Formatear fechas a strings
print(fecha.strftime("%d/%m/%Y")) # 15/03/2024
print(fecha.strftime("%B %Y"))    # March 2024

```

Trabajar con archivos

Leer CSV manualmente

```

import csv

with open("codigos/datos_ventas.csv", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    ventas = list(reader)

print(f"Total registros: {len(ventas)}")
print(f"Primera venta: {ventas[0]}")

# Calcular total de ventas
total = sum(float(v["total"]) for v in ventas)
print(f"Total ventas: {total:,.2f}€")

```

Escribir CSV

```

with open("resumen.csv", "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.writer(f)
    writer.writerow(["Producto", "Unidades", "Ingresos"])
    writer.writerow(["Portátil Pro", 145, 188500])
    writer.writerow(["Smartphone Air", 198, 178200])

```

Crear tu propio módulo

Crea un archivo `analizador.py`:

```
# analizador.py
def calcular_ingresos(precio, cantidad):
    return precio * cantidad

def calcular_margen(venta, coste):
    return (venta - coste) / venta * 100

def clasificar_producto(precio):
    if precio > 1000:
        return "Gama alta"
    elif precio > 300:
        return "Gama media"
    else:
        return "Gama baja"
```

Luego úsalo desde otro archivo:

```
from analizador import calcular_ingresos, clasificar_producto

ingresos = calcular_ingresos(1299.99, 5)
print(f"Ingresos: {ingresos:.2f}€")

gama = clasificar_producto(1299.99)
print(f"Clasificación: {gama}")
```

Ejercicios del Capítulo 14

1. Crea una función `calcular\_iva(precio, tasa=0.21)` que devuelva el precio con IVA.
2. Crea una función `resumen\_ventas(ventas\_lista)` que reciba una lista de totales y devuelva un diccionario con: total, promedio, máximo, mínimo, conteo.
3. Usa el módulo `datetime` para calcular cuántos días han pasado entre dos fechas.
4. Lee el archivo `datos\_ventas.csv` y calcula las ventas totales por región usando un diccionario.
5. Crea tu propio módulo `metrics.py` con funciones de análisis (media, mediana, desviación).
6. Usa el módulo `random` para simular 10 ventas diarias de un producto.
7. Escribe un archivo CSV con los resultados del ejercicio 4.
8. Crea una función que reciba un nombre de archivo CSV y devuelva las primeras 5 filas.
9. Usa `datetime` para calcular la edad de un cliente basada en su fecha de registro.
10. Crea una función `generar\_reporte(archivo\_csv)` que lea un CSV, calcule métricas y las muestre formateadas.

Checklist de autoevaluación

- [] Sé definir funciones con def
- [] Sé usar parámetros, argumentos y return
- [] Sé usar parámetros por defecto

- [] Sé importar módulos estándar (math, datetime, csv)
- [] Sé leer y escribir archivos CSV
- [] Sé crear y usar mis propios módulos
- [] Sé escribir docstrings para documentar funciones

Capítulo 15: Excel — DAX y Power Pivot

¿Qué es DAX?

DAX (Data Analysis Expressions) es el lenguaje de fórmulas de Power Pivot, Power BI y Analysis Services. Es como las fórmulas de Excel pero mucho más potente.

Mientras que las fórmulas de Excel trabajan celda por celda, DAX trabaja sobre tablas enteras y relaciones.

Power Pivot: el motor de análisis

Power Pivot es un motor de análisis en memoria que permite trabajar con millones de filas de múltiples orígenes.

1. "Datos" > "Administrar modelo de datos" (o "Power Pivot" > "Administrar")
2. Si no ves la pestaña Power Pivot: "Archivo" > "Opciones" > "Complementos" > Activar Power Pivot

Cargar datos en Power Pivot

1. En Power Pivot, "Inicio" > "Obtener datos"
2. Carga `datos\_ventas.csv` y `datos\_productos.csv`
3. Crea relaciones entre tablas:
 - Power Pivot > "Diseño" > "Crear relación"
 - Tabla1: `ventas[product\_id]`, Tabla2: `productos[id]`

DAX vs Excel: diferencias clave

Concepto	Excel	DAX
Contexto	Celda actual	Fila actual + filtros
Tablas	Rangos de celdas	Tablas con relaciones
Filtros	Manual	Automático por contexto
Resultado	Una celda	Puede ser una tabla

Medidas vs Columnas calculadas

Columna calculada: evaluada fila por fila

Se crea en Power Pivot > "Agregar columna". Se guarda en la tabla.

```
= ventas[cantidad] * ventas[precio_unitario]
```

Medida: calculada en contexto de filtro

Se crea en Power Pivot > "Nueva medida". Solo existe como agregación.

```
Ingresos Totales = SUMX(ventas, ventas[cantidad] * ventas[precio_unitario])
```

Regla general: usa columnas calculadas para valores por fila, medidas para agregaciones.

Medidas DAX básicas

```
-- Medidas básicas
Total Pedidos = COUNTROWS(ventas)
Total Ingresos = SUM(ventas[total])
Ticket Promedio = AVERAGE(ventas[total])
Pedido Máximo = MAX(ventas[total])
Pedido Mínimo = MIN(ventas[total])

-- Conteo de valores distintos
Clientes Activos = DISTINCTCOUNT(ventas[customer_id])
Productos Vendidos = DISTINCTCOUNT(ventas[product_id])
```

CALCULATE: la función más importante

`CALCULATE` cambia el contexto de filtro de una medida. Es la función más potente de DAX.

```
-- Ventas totales (sin filtro)
Ventas Totales = SUM(ventas[total])

-- Ventas solo de Madrid
Ventas Madrid = CALCULATE(
    SUM(ventas[total]),
    clientes[region] = "Madrid"
)

-- Ventas de productos caros (> 500€)
Ventas Productos Caros = CALCULATE(
    SUM(ventas[total]),
    productos[precio] > 500
)

-- Ventas del año 2025
Ventas 2025 = CALCULATE(
    SUM(ventas[total]),
    YEAR(ventas[fecha]) = 2025
)
```

Múltiples filtros en CALCULATE

```
-- Ventas de Madrid en 2025 de productos caros
Ventas Madrid Premium = CALCULATE(
    SUM(ventas[total]),
    clientes[region] = "Madrid",
    YEAR(ventas[fecha]) = 2025,
```

```

    productos[precio] > 500
)

```

Funciones de iteración (X functions)

SUMX, AVERAGEX, COUNTX, etc. iteran fila por fila:

```

-- SUMX: suma fila por fila
Margen Total = SUMX(
    ventas,
    ventas[cantidad] * (ventas[precio_unitario] - RELATED(productos[coste]))
)

-- AVERAGEX: promedio fila por fila
Ticket Promedio con Detalle = AVERAGEX(
    ventas,
    ventas[cantidad] * ventas[precio_unitario]
)

```

Funciones de fecha y tiempo

```

-- Ventas del período actual
Ventas YTD = TOTALYTD(SUM(ventas[total]), calendario[fecha])
Ventas QTD = TOTALQTD(SUM(ventas[total]), calendario[fecha])
Ventas MTD = TOTALMTD(SUM(ventas[total]), calendario[fecha])

-- Ventas del mismo período año anterior
Ventas Año Anterior = CALCULATE(
    SUM(ventas[total]),
    SAMEPERIODLASTYEAR(calendario[fecha])
)

-- Crecimiento interanual
Crecimiento YoY =
    DIVIDE(
        SUM(ventas[total]) - [Ventas Año Anterior],
        [Ventas Año Anterior]
    )

```

RELATED: navegar relaciones

Similar a BUSCARV en Excel:

```

-- Acceder al precio del producto desde la tabla ventas
Precio Producto = RELATED(productos[precio])

-- Margen por línea de venta
Margen Línea = ventas[cantidad] * (ventas[precio_unitario] - RELATED(productos[coste]))

```

Tabla de calendario

Para análisis temporales, necesitas una tabla de calendario:

```
Calendario = CALENDAR(  
    DATE(2023, 1, 1),  
    DATE(2026, 12, 31)  
)
```

Luego añade columnas:

```
Año = YEAR(calendario[fecha])  
Mes = FORMAT(calendario[fecha], "MMMM")  
Mes Número = MONTH(calendario[fecha])  
Trimestre = "Q" & QUARTER(calendario[fecha])  
Año-Mes = FORMAT(calendario[fecha], "YYYY-MM")
```

Medida de ejemplo: dashboard completo

```
-- KPIs principales  
Ventas Totales = SUM(ventas[total])  
Pedidos = COUNTROWS(ventas)  
Ticket Promedio = AVERAGE(ventas[total])  
Clientes = DISTINCTCOUNT(ventas[customer_id])  
  
-- Ventas por región  
Ventas Madrid = CALCULATE([Ventas Totales], clientes[region] = "Madrid")  
% Madrid = DIVIDE([Ventas Madrid], [Ventas Totales])  
  
-- Crecimiento  
Ventas Año Anterior = CALCULATE([Ventas Totales], SAMEPERIODLASTYEAR(calendario[fecha]))  
Crecimiento vs Año Ant = DIVIDE([Ventas Totales] - [Ventas Año Anterior], [Ventas Año Anterior])
```

Ejercicios del Capítulo 15

1. Carga `datos\_ventas.csv` en Power Pivot y crea una medida `Total Ventas` que sume `total`.
2. Crea una medida `Total Pedidos` que cuente filas de la tabla ventas.
3. Crea una medida `Ventas Cataluña` que calcule ventas solo de Cataluña.
4. Carga `datos\_productos.csv` y relaciónala con ventas por `product\_id`.
5. Crea una medida `Margen Total` usando SUMX y RELATED para obtener el coste.
6. Crea una tabla de calendario con CALENDAR para el rango 2023-2026.
7. Crea medidas de Ventas YTD y compáralas con el año anterior.
8. Crea una medida de crecimiento interanual como porcentaje.
9. Diseña un KPI card en Power Pivot o Power BI con: ventas totales, pedidos, ticket promedio.
10. Crea una medida que calcule el % de ventas de cada región sobre el total.

Checklist de autoevaluación

- ☐ Sé cargar datos en Power Pivot
- ☐ Sé crear relaciones entre tablas
- ☐ Entiendo la diferencia entre medidas y columnas calculadas

- [] Sé escribir medidas DAX básicas (SUM, COUNT, AVERAGE)
- [] Sé usar CALCULATE para modificar contexto
- [] Sé usar SUMX y otras funciones de iteración
- [] Sé usar funciones de fecha y tiempo (YTD, SAMEPERIODLASTYEAR)
- [] Sé crear una tabla de calendario

Checkpoint 3: Análisis de Cohorte de Ventas

Proyecto integrador del Proyecto 3

En este checkpoint vas a realizar un análisis de cohortes para entender el comportamiento de los clientes de TechStore a lo largo del tiempo.

¿Qué es un análisis de cohortes?

Un análisis de cohortes agrupa clientes por el período en que hicieron su primera compra y analiza su comportamiento en meses posteriores. Responde preguntas como:

- ¿Los clientes que se registraron en 2024 son más fieles que los de 2025?
- ¿Cuánto gastan los clientes en su segundo mes vs su primer mes?
- ¿Qué cohorte tiene mejor retención?

Parte 1: SQL — Prepara los datos de cohortes

```
-- 3.1 Primera compra de cada cliente
WITH primera_compra AS (
    SELECT customer_id,
           MIN(order_date) AS fecha_primera_compra,
           strftime('%Y-%m', MIN(order_date)) AS cohorte
    FROM orders
    WHERE status = 'Completado'
    GROUP BY customer_id
),
-- 3.2 Compras por cliente y mes
compras_mensuales AS (
    SELECT o.customer_id,
           pc.cohorte,
           strftime('%Y-%m', o.order_date) AS mes_compra,
           (strftime('%Y', o.order_date) - strftime('%Y', pc.fecha_primera_compra)) * 12
           + (strftime('%m', o.order_date) - strftime('%m', pc.fecha_primera_compra)) AS mes_relativo,
           COUNT(DISTINCT o.id) AS pedidos,
           SUM(o.total) AS gasto
    FROM orders o
    JOIN primera_compra pc ON o.customer_id = pc.customer_id
    WHERE o.status = 'Completado'
    GROUP BY o.customer_id, pc.cohorte, mes_compra
```

```

)
-- 3.3 Tabla de cohortes
SELECT cohorte,
       mes_relativo,
       COUNT(DISTINCT customer_id) AS clientes_activos,
       ROUND(AVG(gasto), 2) AS gasto_promedio,
       ROUND(SUM(gasto), 2) AS gasto_total
FROM compras_mensuales
GROUP BY cohorte, mes_relativo
ORDER BY cohorte, mes_relativo;

```

```

-- 3.4 Ventas totales por mes
SELECT strftime('%Y-%m', order_date) AS mes,
       COUNT(*) AS pedidos,
       COUNT(DISTINCT customer_id) AS clientes,
       ROUND(SUM(total), 2) AS ingresos
FROM orders
WHERE status = 'Completado'
GROUP BY mes
ORDER BY mes;

```

```

-- 3.5 Distribución de valor de pedidos
SELECT CASE
       WHEN total < 50 THEN '< 50€'
       WHEN total < 100 THEN '50-100€'
       WHEN total < 200 THEN '100-200€'
       WHEN total < 500 THEN '200-500€'
       WHEN total < 1000 THEN '500-1000€'
       ELSE '+1000€'
       END AS rango,
       COUNT(*) AS pedidos,
       ROUND(SUM(total), 2) AS ingresos,
       ROUND(AVG(total), 2) AS ticket_promedio
FROM orders WHERE status = 'Completado'
GROUP BY rango
ORDER BY MIN(total);

```

```

-- 3.6 Frecuencia de compra por cliente
SELECT pedidos_realizados,
       COUNT(*) AS clientes
FROM (
       SELECT customer_id, COUNT(*) AS pedidos_realizados
       FROM orders WHERE status = 'Completado'
       GROUP BY customer_id
)
GROUP BY pedidos_realizados
ORDER BY pedidos_realizados;

```

Parte 2: Excel — Power Query y tablas dinámicas

1. Carga `datos\_detalle\_ventas.csv` en Power Query
2. Realiza las siguientes transformaciones:

- Cambia tipo de `order\_date` a fecha
 - Filtra solo pedidos "Completado"
 - Agrupa por `customer\_id` y mes (usa columna fecha agrupada)
 - Carga el resultado a una tabla
3. Crea una tabla dinámica con:
 - Filas: mes_relativo
 - Columnas: cohorte
 - Valores: clientes activos (conteo de customer_id)
 4. Crea un gráfico de líneas para visualizar las cohortes
 5. Utiliza formato condicional en la tabla dinámica para ver las cohortes con más actividad

Parte 3: Python — Análisis de cohortes automatizado

```
import csv
from datetime import datetime
from collections import defaultdict

def analisis_cohortes():
    # Leer ventas
    ventas = []
    with open("codigos/datos_ventas.csv", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            if row["status"] == "Completado":
                ventas.append({
                    "customer_id": int(row["customer_id"]),
                    "order_date": row["order_date"],
                    "total": float(row["total"]),
                    "region": row["region"]
                })

    # Encontrar primera compra de cada cliente
    primera_compra = {}
    for v in ventas:
        cid = v["customer_id"]
        fecha = v["order_date"][:7] # YYYY-MM
        if cid not in primera_compra or fecha < primera_compra[cid]:
            primera_compra[cid] = fecha

    # Asignar cohorte a cada venta
    cohortes = defaultdict(lambda: defaultdict(lambda: {"clientes": set(), "gasto": 0.0, "pedidos": 0}))

    for v in ventas:
        cid = v["customer_id"]
        cohorte = primera_compra[cid]
        mes_compra = v["order_date"][:7]

        # Calcular mes relativo
        año_c, mes_c = int(cohorte[:4]), int(cohorte[5:7])
```



```

año_v, mes_v = int(mes_compra[:4]), int(mes_compra[5:7])
mes_rel = (año_v - año_c) * 12 + (mes_v - mes_c)

cohortes[cohortes][mes_rel]["clientes"].add(cid)
cohortes[cohortes][mes_rel]["gasto"] += v["total"]
cohortes[cohortes][mes_rel]["pedidos"] += 1

# Mostrar matriz de cohortes
print("=== ANÁLISIS DE COHORTES ===\n")
print("Clientes activos por mes relativo:\n")

cohortes_ordenadas = sorted(cohortes.keys())
meses_max = max(max(meses.keys()) for meses in cohortes.values())

# Cabecera
header = "Cohorte  "
for m in range(meses_max + 1):
    header += f" Mes {m:2d} "
print(header)
print("=" * len(header))

for cohorte in cohortes_ordenadas:
    row = f"{cohorte}  "
    for m in range(meses_max + 1):
        num = len(cohortes[cohorte][m]["clientes"]) if m in cohortes[cohorte] else 0
        row += f" {num:4d} "
    print(row)

# Métricas de retención
print("\n=== RETENCIÓN POR COHORTE ===\n")
print(f"{'Cohorte':<10} {'Mes 0':>8} {'Mes 1':>8} {'Mes 3':>8} {'Mes 6':>8} {'Mes 12':>8}")
print("=" * 55)

for cohorte in cohortes_ordenadas:
    base = len(cohortes[cohorte][0]["clientes"])
    if base == 0:
        continue
    m0 = 100.0
    m1 = (len(cohortes[cohorte][1]["clientes"]) / base * 100) if 1 in cohortes[cohorte] else 0
    m3 = (len(cohortes[cohorte][3]["clientes"]) / base * 100) if 3 in cohortes[cohorte] else 0
    m6 = (len(cohortes[cohorte][6]["clientes"]) / base * 100) if 6 in cohortes[cohorte] else 0
    m12 = (len(cohortes[cohorte][12]["clientes"]) / base * 100) if 12 in cohortes[cohorte] else 0
    print(f"{cohorte:<10} {m0:>7.1f}% {m1:>7.1f}% {m3:>7.1f}% {m6:>7.1f}% {m12:>7.1f}%")

# Métricas globales
total_clientes = len(set(v["customer_id"] for v in ventas))
total_ingresos = sum(v["total"] for v in ventas)

print(f"\n=== RESUMEN ===")
print(f"Total clientes activos: {total_clientes}")
print(f"Total ingresos: {total_ingresos:,.2f}€")
print(f"Ingreso por cliente: {total_ingresos/total_clientes:,.2f}€")

```

```

# Distribución por frecuencia
freq = defaultdict(int)
for v in ventas:
    freq[v["customer_id"]] += 1

unicos = set(v["customer_id"] for v in ventas)
una_vez = sum(1 for c in unicos if freq[c] == 1)
print(f"\nClientes que compraron solo una vez: {una_vez} ({una_vez/len(unicos)*100:.1f}%)")

 analisis_cohortes()

```

Parte 4: Preguntas de negocio

1. ¿Qué cohorte tiene mejor retención en el mes 3? ¿Por qué crees?
2. ¿Cuánto gastan los clientes en promedio en su primera compra vs la segunda?
3. ¿Hay estacionalidad en las ventas? ¿Qué meses venden más?
4. ¿Qué porcentaje de clientes compra más de una vez?
5. ¿Qué recomendarías al equipo de marketing basado en este análisis?

Entregable

Guarda tu análisis en `proyecto\_3/`:

- ` analisis\_cohortes.sql ` — Consultas SQL
- ` cohortes.xlsx ` — Tabla dinámica con matriz de cohortes
- ` analisis\_cohortes.py ` — Análisis Python automatizado
- ` dashboard\_cohortes.png ` — Captura de tu dashboard

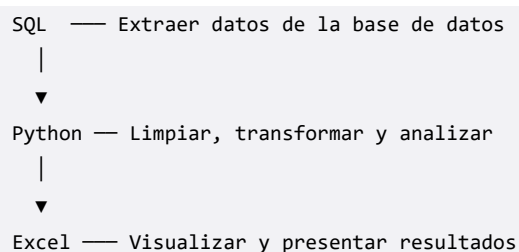
Checklist de autoevaluación del Proyecto 3

- [] Sé usar subconsultas y CTEs para análisis complejos
- [] Sé usar funciones de ventana (ROW_NUMBER, LAG, NTILE)
- [] Sé usar Power Query para transformar datos
- [] Sé crear medidas DAX con CALCULATE y SUMX
- [] Sé crear funciones en Python y usar módulos
- [] Sé leer archivos CSV con Python
- [] Completé el análisis de cohortes en las 3 herramientas

Capítulo 16: Python + SQL + Excel — La Integración Total

El flujo completo del analista

En proyectos reales, un analista combina las tres herramientas en un flujo de trabajo:



En este capítulo aprenderás a conectar estas herramientas.

Conectar Python con SQLite

```
import sqlite3
import csv
from datetime import datetime

# Conectar a la base de datos
conn = sqlite3.connect("codigos/techstore.db")
conn.row_factory = sqlite3.Row # Permite acceder por nombre de columna

# Ejecutar consulta
cursor = conn.execute("""
    SELECT c.region,
           strftime('%Y-%m', o.order_date) AS mes,
           COUNT(DISTINCT o.id) AS pedidos,
           ROUND(SUM(o.total), 2) AS ingresos
    FROM orders o
    JOIN customers c ON o.customer_id = c.id
    WHERE o.status = 'Completado'
    GROUP BY c.region, mes
    ORDER BY c.region, mes
""")

# Leer resultados
```

```

filas = cursor.fetchall()
print(f"Total filas: {len(filas)}")
print(f"Columnas: {[desc[0] for desc in cursor.description]}")

# Convertir a lista de diccionarios
resultados = [dict(fila) for fila in filas]

# Mostrar primeras 5
for r in resultados[:5]:
    print(r)

conn.close()

```

pandas: la biblioteca estrella

pandas es la biblioteca más importante para análisis de datos en Python. Trabaja con DataFrames (tablas en memoria).

```

# pip install pandas
import pandas as pd

# Leer CSV directamente
df = pd.read_csv("codigos/datos_ventas.csv")
print(df.head())          # Primeras 5 filas
print(df.shape)           # Dimensiones (filas, columnas)
print(df.dtypes)          # Tipos de datos

# Leer SQL directamente
conn = sqlite3.connect("codigos/techstore.db")
df_ventas = pd.read_sql("""
    SELECT o.*, c.region, c.city
    FROM orders o
    JOIN customers c ON o.customer_id = c.id
""", conn)
conn.close()

print(df_ventas.head())

```

Operaciones básicas con pandas

```

# Filtros
df_madrid = df_ventas[df_ventas["region"] == "Madrid"]
df_caros = df_ventas[df_ventas["total"] > 1000]

# Agrupación
resumen = df_ventas.groupby("region").agg(
    pedidos=("id", "count"),
    ingresos=("total", "sum"),
    ticket_promedio=("total", "mean")
).reset_index()

print(resumen)

```

```
# Ordenar
resumen = resumen.sort_values("ingresos", ascending=False)

# Añadir columna calculada
df_ventas["año"] = pd.to_datetime(df_ventas["order_date"]).dt.year
df_ventas["mes"] = pd.to_datetime(df_ventas["order_date"]).dt.month
```

Exportar de Python a Excel

```
# Crear un archivo Excel con múltiples hojas
with pd.ExcelWriter("reporte_techstore.xlsx", engine="openpyxl") as writer:
    # Hoja 1: Resumen por región
    resumen.to_excel(writer, sheet_name="Resumen Regiones", index=False)

    # Hoja 2: Top productos
    df_detalle = pd.read_csv("codigos/datos_detalle_ventas.csv")
    top_productos = df_detalle.groupby("product_name").agg(
        unidades=("quantity", "sum"),
        ingresos=("line_total", "sum")
    ).sort_values("unidades", ascending=False).head(20).reset_index()
    top_productos.to_excel(writer, sheet_name="Top Productos", index=False)

    # Hoja 3: Ventas mensuales
    df_ventas["order_date"] = pd.to_datetime(df_ventas["order_date"])
    df_ventas["mes"] = df_ventas["order_date"].dt.to_period("M")
    mensual = df_ventas.groupby("mes").agg(
        pedidos=("id", "count"),
        ingresos=("total", "sum")
    ).reset_index()
    mensual["mes"] = mensual["mes"].astype(str)
    mensual.to_excel(writer, sheet_name="Ventas Mensuales", index=False)

print("Reporte Excel generado: reporte_techstore.xlsx")
```

Script completo: pipeline automatizado

```
"""
pipeline_analisis.py
Pipeline completo de análisis: SQL → pandas → Excel
"""

import sqlite3
import pandas as pd
from datetime import datetime

def extraer_datos(db_path):
    """Extraer datos de SQLite"""
    conn = sqlite3.connect(db_path)

    query = """
```

```

        SELECT o.id AS order_id, o.order_date, o.status, o.total,
               c.first_name || ' ' || c.last_name AS cliente,
               c.region, c.city,
               e.first_name || ' ' || e.last_name AS vendedor
        FROM orders o
        JOIN customers c ON o.customer_id = c.id
        JOIN employees e ON o.employee_id = e.id
    """

    df = pd.read_sql(query, conn)
    conn.close()
    return df

def transformar_datos(df):
    """Limpiar y transformar"""
    df["order_date"] = pd.to_datetime(df["order_date"])
    df["año"] = df["order_date"].dt.year
    df["mes"] = df["order_date"].dt.month
    df["día_semana"] = df["order_date"].dt.day_name()
    return df

def generar_reportes(df, output_path):
    """Generar reportes Excel"""
    with pd.ExcelWriter(output_path, engine="openpyxl") as writer:
        # Ventas por región
        df[df["status"] == "Completado"].groupby("region").agg(
            pedidos=("order_id", "count"),
            ingresos=("total", "sum"),
            ticket_promedio=("total", "mean")
        ).sort_values("ingresos", ascending=False).to_excel(
            writer, sheet_name="Por Región", index=True
        )

        # Ventas por año-mes
        df[df["status"] == "Completado"].groupby(
            df["order_date"].dt.to_period("M")
        ).agg(
            pedidos=("order_id", "count"),
            ingresos=("total", "sum")
        ).to_excel(writer, sheet_name="Tendencia Mensual", index=True)

        # Top 10 vendedores
        df[df["status"] == "Completado"].groupby("vendedor").agg(
            pedidos=("order_id", "count"),
            ingresos=("total", "sum")
        ).sort_values("ingresos", ascending=False).head(10).to_excel(
            writer, sheet_name="Top Vendedores", index=True
        )

    if __name__ == "__main__":
        print("=== Pipeline de Análisis TechStore ===")
        print("Extrayendo datos...")
        df = extraer_datos("codigos/techstore.db")

```

```

print(f" {len(df)} registros extraídos")

print("Transformando datos...")
df = transformar_datos(df)

print("Generando reportes...")
output = f"reporte_techstore_{datetime.now().strftime('%Y%m%d')}.xlsx"
generar_reportes(df, output)
print(f" Reporte generado: {output}")
print("|Pipeline completado!")

```

Ejercicios del Capítulo 16

1. Conecta Python a la base de datos TechStore y extrae todos los clientes.
2. Usa pandas para leer `datos\_ventas.csv` y muestra las primeras 10 filas.
3. Calcula el total de ventas por región usando pandas groupby.
4. Exporta el resultado del ejercicio 3 a un archivo Excel.
5. Crea un script que lea datos de SQLite, los transforme y los guarde en Excel.
6. Usa pandas para filtrar pedidos con total > 500 y cuenta cuántos hay.
7. Añade una columna "trimestre" a un DataFrame de ventas.
8. Exporta un Excel con 3 hojas: resumen general, top clientes, y ventas mensuales.
9. Crea una función que reciba una consulta SQL y devuelva un DataFrame.
10. Combina datos de ventas y productos usando pandas merge (equivalente a JOIN).

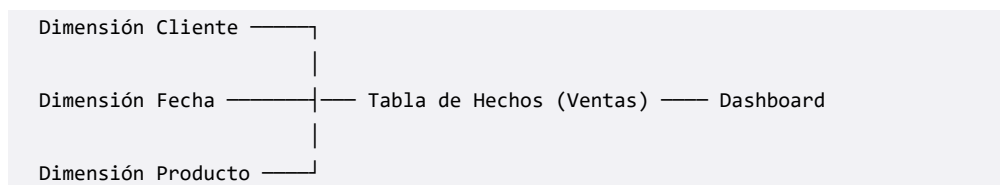
Checklist de autoevaluación

- ☐ Sé conectar Python con SQLite usando sqlite3
- ☐ Sé usar pandas para leer CSV y SQL
- ☐ Sé filtrar, agrupar y ordenar con pandas
- ☐ Sé exportar DataFrames a Excel con múltiples hojas
- ☐ Sé crear un pipeline automatizado SQL pandas Excel
- ☐ Entiendo el flujo de trabajo completo del analista

Capítulo 17: Modelo de Datos y Dashboard Profesional

Modelado dimensional

El modelado dimensional organiza los datos en **hechos** (métricas) y **dimensiones** (contexto). Es el estándar en business intelligence.



En Power Pivot / Power BI:

- **Tabla de hechos:** `ventas` (order_id, customer_id, product_id, date_id, cantidad, precio, total)
- **Dimensiones:** `clientes`, `productos`, `calendario`

Construir el modelo en Power Pivot

1. Carga las siguientes tablas en Power Pivot:
 - `datos\_detalle\_ventas.csv` (hechos)
 - `datos\_productos.csv` (dimensión)
 - `datos\_clientes.csv` (dimensión)
 - Crea una tabla `calendario` con CALENDAR
2. Crea relaciones:

Tabla origen	Columna	Tabla destino	Columna
detalle_ventas	customer_id	clientes	id
detalle_ventas	product_id	productos	id
detalle_ventas	order_date	calendario	fecha

Medidas DAX para el dashboard

```
-- KPIs principales
Ventas Totales = SUM(detalle_ventas[line_total])
```



```

Pedidos = DISTINCTCOUNT(detalles_ventas[order_id])
Clientes Activos = DISTINCTCOUNT(detalles_ventas[customer_id])
Productos Vendidos = DISTINCTCOUNT(detalles_ventas[product_name])
Ticket Promedio = DIVIDE([Ventas Totales], [Pedidos])
Unidades Vendidas = SUM(detalles_ventas[quantity])

-- Métricas de comparación
Ventas Año Anterior = CALCULATE(
    [Ventas Totales],
    SAMEPERIODLASTYEAR(calendario[fecha])
)
Crecimiento % = DIVIDE(
    [Ventas Totales] - [Ventas Año Anterior],
    [Ventas Año Anterior]
)

-- Ventas por categoría
Ventas Portátiles = CALCULATE(
    [Ventas Totales],
    productos[category_name] = "Portátiles"
)

-- Ranking de productos
Top Productos = TOPN(10, productos, [Ventas Totales])

```

Diseño del dashboard profesional

Un dashboard debe responder 3 preguntas en 3 segundos:

1. **¿Cómo vamos?** (KPIs principales)
2. **¿Qué está pasando?** (tendencias, comparativas)
3. **¿Qué debo hacer?** (insights, alertas)

Layout recomendado

TECHSTORE - DASHBOARD DE VENTAS			
Ventas	Pedidos	Clientes	Ticket Promedio
€1.2M	5,250	245	€228
vs PY +12%			
Ventas por Región (Gráfico de barras)		Ventas Mensuales (Gráfico línea)	
Top 5 Productos (Gráfico barras)		Ventas por Categoría (pastel)	

Crear el dashboard paso a paso

1. En Power Pivot, crea las medidas del dashboard
2. Inserta tablas dinámicas y gráficos dinámicos
3. Conecta todos los elementos a los mismos segmentadores
4. Organiza en una hoja "Dashboard"

Segmentadores compartidos

1. Crea un segmentador de año (de la tabla calendario)
2. Crea un segmentador de región (de la tabla clientes)
3. Conecta cada segmentador a todas las tablas dinámicas:
 - Haz clic derecho en el segmentador
 - "Conexiones de informes"
 - Selecciona todas las tablas dinámicas

Formato profesional

Tarjetas KPI

1. Crea una tabla dinámica con una sola celda para cada KPI
2. Formatea el número grande (18-24pt)
3. Añade la etiqueta debajo
4. Añade formato condicional: verde si crecimiento > 0, rojo si < 0

Colores

KPI positivo: #00B050 (verde)
KPI negativo: #FF0000 (rojo)
Fondo dashboard: #F2F2F2 (gris claro)
Gráficos: paleta azul corporativo

Reglas de diseño

1. **Máximo 5 KPIs** en la fila superior
2. **Máximo 4 gráficos** en el cuerpo
3. **Mismo tipo de gráfico** = mismo color
4. **Ordena los datos** de mayor a menor en barras
5. **Elimina leyendas** si solo hay una serie
6. **Títulos descriptivos**: "Ventas por Región (2025)" no "Gráfico 1"

Validación del dashboard

Antes de compartir, pregúntate:

1. **Total correcto**: ¿Suma todas las regiones el total general?
2. **Filtros**: ¿Funcionan todos los segmentadores?
3. **Consistencia**: ¿Coinciden los números con los del reporte anterior?
4. **Rendimiento**: ¿Tarda mucho en actualizar?

5. **Claridad:** ¿Lo entendería alguien que no sabe de datos?

Ejercicios del Capítulo 17

1. Carga las tablas en Power Pivot y crea las relaciones correctas.
2. Crea una tabla de calendario con CALENDAR para el rango 2023-2026.
3. Crea las medidas: Ventas Totales, Pedidos, Clientes Activos, Ticket Promedio.
4. Crea una medida de crecimiento interanual.
5. Diseña un layout de dashboard con 4 KPIs arriba y 4 gráficos abajo.
6. Crea un gráfico de ventas por región (barras) y otro de tendencia mensual (línea).
7. Conecta segmentadores de año y región a todos los elementos.
8. Aplica formato condicional a los KPIs (verde/rojo según crecimiento).
9. Añade una tabla de Top 10 productos más vendidos.
10. Valida el dashboard: comprueba que los totales coinciden con SQL.

Checklist de autoevaluación

- ☐ Entiendo el modelo dimensional (hechos y dimensiones)
- ☐ Sé crear relaciones entre tablas en Power Pivot
- ☐ Sé crear medidas DAX para KPIs
- ☐ Sé diseñar un dashboard con layout profesional
- ☐ Sé conectar segmentadores a múltiples elementos
- ☐ Sé aplicar formato condicional y colores corporativos
- ☐ Sé validar un dashboard antes de compartirlo

Capítulo 18: Proyecto Integrador Final

El desafío completo

Has llegado al final. Este proyecto integra todo lo aprendido en los 17 capítulos anteriores. Vas a construir un pipeline completo de análisis que va desde la base de datos hasta un dashboard profesional.

Escenario

La dirección de TechStore quiere un **sistema de reporting automatizado** que:

1. Se actualice automáticamente con nuevos datos
2. Proporcione una visión general del negocio
3. Identifique tendencias y anomalías
4. Sea fácil de compartir con el equipo

Fase 1: Extracción con SQL

Crea un archivo `consultas\_finales.sql` con estas consultas:

```
-- 1.1 Resumen general del negocio
SELECT 'Total Ventas' AS metrica, ROUND(SUM(total), 2) AS valor FROM orders WHERE status = 'Completado'
UNION ALL
SELECT 'Total Pedidos', CAST(COUNT(*) AS REAL) FROM orders WHERE status = 'Completado'
UNION ALL
SELECT 'Clientes Únicos', CAST(COUNT(DISTINCT customer_id) AS REAL) FROM orders WHERE status = 'Completado'
UNION ALL
SELECT 'Ticket Promedio', ROUND(AVG(total), 2) FROM orders WHERE status = 'Completado'
UNION ALL
SELECT 'Productos Vendidos', CAST(SUM(oi.quantity) AS REAL)
FROM order_items oi JOIN orders o ON oi.order_id = o.id WHERE o.status = 'Completado'
UNION ALL
SELECT 'Tasa Cancelación', ROUND(
    CAST(SUM(CASE WHEN status = 'Cancelado' THEN 1 ELSE 0 END) AS REAL) / COUNT(*) * 100, 1
) FROM orders;

-- 1.2 Ventas por región y categoría
SELECT c.region, cat.name AS categoria,
    COUNT(DISTINCT o.id) AS pedidos,
    SUM(oi.quantity) AS unidades,
    ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM customers c
```

```

JOIN orders o ON c.id = o.customer_id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
JOIN categories cat ON p.category_id = cat.id
WHERE o.status = 'Completado'
GROUP BY c.region, cat.name
ORDER BY c.region, ingresos DESC;

-- 1.3 Top 20 productos
SELECT p.name AS producto, cat.name AS categoria,
       SUM(oi.quantity) AS unidades,
       ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
FROM products p
JOIN categories cat ON p.category_id = cat.id
JOIN order_items oi ON p.id = oi.product_id
JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'Completado'
GROUP BY p.id
ORDER BY ingresos DESC
LIMIT 20;

-- 1.4 Rendimiento de vendedores
SELECT e.first_name || ' ' || e.last_name AS vendedor,
       e.position AS cargo,
       COUNT(DISTINCT o.id) AS pedidos,
       ROUND(SUM(o.total), 2) AS ingresos,
       ROUND(AVG(o.total), 2) AS ticket_promedio
FROM employees e
JOIN orders o ON e.id = o.employee_id
WHERE o.status = 'Completado'
GROUP BY e.id
ORDER BY ingresos DESC;

-- 1.5 Evolución mensual (2024-2026)
SELECT strftime('%Y-%m', order_date) AS mes,
       COUNT(*) AS pedidos,
       COUNT(DISTINCT customer_id) AS clientes,
       ROUND(SUM(total), 2) AS ingresos
FROM orders
WHERE status = 'Completado'
GROUP BY mes
ORDER BY mes;

```

Fase 2: Pipeline Python

Crea `pipeline\_final.py`:

```

"""
pipeline_final.py
Pipeline automatizado TechStore
SQL → pandas → Excel → Dashboard
"""

```

```

import sqlite3
import pandas as pd
from datetime import datetime
import os

def conectar_bd(ruta="codigos/techstore.db"):
    return sqlite3.connect(ruta)

def extraer_metricas(conn):
    return pd.read_sql("""
        SELECT 'Ventas Totales' AS metrica, ROUND(SUM(total), 2) AS valor
        FROM orders WHERE status = 'Completado'
    """, conn)

def extraer_ventas_mensuales(conn):
    return pd.read_sql("""
        SELECT strftime('%Y-%m', order_date) AS mes,
            COUNT(*) AS pedidos,
            COUNT(DISTINCT customer_id) AS clientes,
            ROUND(SUM(total), 2) AS ingresos
        FROM orders WHERE status = 'Completado'
        GROUP BY mes ORDER BY mes
    """, conn)

def extraer_top_productos(conn):
    return pd.read_sql("""
        SELECT p.name AS producto, SUM(oi.quantity) AS unidades,
            ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
        FROM products p
        JOIN order_items oi ON p.id = oi.product_id
        JOIN orders o ON oi.order_id = o.id
        WHERE o.status = 'Completado'
        GROUP BY p.id ORDER BY ingresos DESC LIMIT 10
    """, conn)

def generar_excel(conn, output="reporte_final_techstore.xlsx"):
    with pd.ExcelWriter(output, engine="openpyxl") as writer:
        extraer_ventas_mensuales(conn).to_excel(writer, sheet_name="Tendencia", index=False)
        extraer_top_productos(conn).to_excel(writer, sheet_name="Top Productos", index=False)

    region_cat = pd.read_sql("""
        SELECT c.region, cat.name AS categoria,
            COUNT(DISTINCT o.id) AS pedidos,
            ROUND(SUM(oi.quantity * oi.unit_price), 2) AS ingresos
        FROM customers c JOIN orders o ON c.id = o.customer_id
        JOIN order_items oi ON o.id = oi.order_id
        JOIN products p ON oi.product_id = p.id
        JOIN categories cat ON p.category_id = cat.id
        WHERE o.status = 'Completado'
        GROUP BY c.region, cat.name
    """, conn)
    region_cat.to_excel(writer, sheet_name="Región-Categoría", index=False)

```

```

        pd.read_sql("SELECT * FROM products ORDER BY price DESC LIMIT 10", conn).to_excel(
            writer, sheet_name="Productos Caros", index=False)

    print(f"Reporte Excel generado: {output}")
    return output

def main():
    print("=" * 50)
    print("PIPELINE FINAL - TECHSTORE")
    print("=" * 50)

    conn = conectar_bd()

    print("\n1. Extrayendo métricas...")
    generar_excel(conn)

    print("\n2. Resumen de ejecución:")
    print(f"    Fecha: {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    print(f"    Base datos: techstore.db")
    print(f"    Reporte: reporte_final_techstore.xlsx")

    conn.close()
    print("\n¡Pipeline completado exitosamente!")

if __name__ == "__main__":
    main()

```

Fase 3: Dashboard en Excel

Crea un dashboard profesional en Excel con:

1. **Hoja "Dashboard"** con:

- 4 tarjetas KPI: Ventas, Pedidos, Clientes, Ticket Promedio
- Gráfico de tendencia mensual (líneas)
- Gráfico de ventas por región (barras)
- Gráfico de Top 10 productos (barras horizontales)
- Gráfico de distribución por categoría (circular)
- Segmentadores de año y región

2. **Hoja "Datos"** con los resultados de tus consultas SQL

3. **Hoja "Metodología"** explicando el análisis

Fase 4: Presentación ejecutiva

Prepara un resumen ejecutivo (5 diapositivas máximo) con:

1. **Resumen:** métricas principales del negocio
2. **Regiones:** qué regiones rinden mejor
3. **Productos:** qué productos destacan

4. **Tendencias:** evolución mensual y estacionalidad
5. **Recomendaciones:** 3 acciones concretas para el negocio

Criterios de evaluación

Tu proyecto final debe demostrar:

- ☐ **SQL:** Consultas correctas con JOINS, agregaciones y filtros
- ☐ **Python:** Script funcional que genera reportes automáticamente
- ☐ **Excel:** Dashboard profesional con KPIs, gráficos y segmentadores
- ☐ **Modelado:** Relaciones correctas entre tablas
- ☐ **DAX:** Medidas correctas para KPIs
- ☐ **Calidad:** Datos validados y consistentes
- ☐ **Presentación:** Dashboard claro y profesional

Checklist de autoevaluación final

- ☐ Completé las 5 consultas SQL del proyecto final
- ☐ El pipeline Python se ejecuta sin errores
- ☐ El reporte Excel contiene todas las hojas requeridas
- ☐ El dashboard tiene 4 KPIs correctamente calculados
- ☐ Los gráficos muestran la información claramente
- ☐ Los segmentadores filtran correctamente
- ☐ Validé que los totales del dashboard coinciden con SQL
- ☐ Preparé la presentación ejecutiva
- ☐ Guardé todo en la carpeta `proyecto\_final/`

Checkpoint 4: Pipeline de Análisis Automatizado

Integración final

Este checkpoint valida que has construido un pipeline completo y funcional que integra SQL, Python y Excel en un flujo automatizado.

Parte 1: Verificación del pipeline

Ejecuta tu pipeline Python completo y verifica:

1. El script se ejecuta sin errores
2. Genera el archivo `reporte\_final\_techstore.xlsx`
3. El archivo tiene todas las hojas esperadas
4. Los datos son consistentes (comprueba totales contra SQL)

Script de verificación

Crea `verificar\_pipeline.py`:

```
"""
verificar_pipeline.py
Verifica que el pipeline funciona correctamente
"""

import sqlite3
import pandas as pd
import os

print("=== VERIFICACIÓN DEL PIPELINE ===\n")

# 1. Verificar conexión a BD
print("1. Conexión a base de datos...")
conn = sqlite3.connect("codigos/techstore.db")
tablas = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", conn)
print(f"  Tablas encontradas: {len(tablas)}")
print(f"  Tablas: {'', '.join(tablas['name'].tolist())}")

# 2. Verificar totales
print("\n2. Verificación de totales...")
total_sql = pd.read_sql("""
    SELECT COUNT(*) AS pedidos,
           ROUND(SUM(total), 2) AS ingresos,
```

```

        ROUND(AVG(total), 2) AS ticket_promedio
    FROM orders WHERE status = 'Completado'
    """, conn)
print(f"    Pedidos completados: {total_sql['pedidos'][0]}")
print(f"    Ingresos totales: {total_sql['ingresos'][0]:,.2f}€")
print(f"    Ticket promedio: {total_sql['ticket_promedio'][0]:,.2f}€")

conn.close()

# 3. Verificar que existe el reporte
print("\n3. Verificación de reporte...")
reporte = "reporte_final_techstore.xlsx"
if os.path.exists(reporte):
    tamaño = os.path.getsize(reporte) // 1024
    print(f"    Reporte encontrado: {reporte} ({tamaño} KB)")

    # Verificar hojas
    xl = pd.ExcelFile(reporte)
    print(f"    Hojas: {xl.sheet_names}")
else:
    print(f"    [FALTA] No se encontró {reporte}")

# 4. Verificar cobertura de herramientas
print("\n4. Cobertura de aprendizaje:")
print("    SQL:    Consultas, JOINS, agregaciones, window functions")
print("    Excel:  Tablas dinámicas, Power Query, gráficos, Power Pivot")
print("    Python: variables, listas, dicts, funciones, pandas")
print("    Integración: SQL + Python + Excel pipeline")

print("\n=== VERIFICACIÓN COMPLETADA ===")

```

Parte 2: Validación cruzada

Verifica que los números coinciden entre herramientas:

```

-- SQL: Total de ventas completadas
SELECT ROUND(SUM(total), 2) FROM orders WHERE status = 'Completado';

```

```

# Python: mismo cálculo con pandas
df = pd.read_sql("SELECT total FROM orders WHERE status = 'Completado'", conn)
print(df["total"].sum())

```

```

# Excel: Tabla dinámica filtrando status = Completado
# Valor: Suma de total
# ¿Coincide con SQL y Python?

```

Los tres deben dar exactamente el mismo número.

Parte 3: Portfolio completo

Al finalizar este libro, tienes un portfolio con 4 proyectos:

Proyecto	Herramientas	Qué hiciste
1. Análisis de Ventas Básicas	SQL + Excel + Python	Primeros pasos, métricas básicas
2. Top 10 Productos	SQL + Excel + Python	JOINS, tablas dinámicas, visualización
3. Análisis de Cohorte	SQL + Excel + Python	CTEs, Power Query, Python avanzado
4. Pipeline Automatizado	SQL + Python + Excel	Integración total, dashboard

Cada proyecto incluye:

- Consultas SQL
- Archivo Excel con análisis
- Script Python
- Dashboard o visualización

Parte 4: Checklist final del libro

- ☐ Ejecuté el pipeline final sin errores
- ☐ Verifiqué que los totales coinciden en las 3 herramientas
- ☐ Mi dashboard responde las preguntas de negocio
- ☐ Tengo los 4 proyectos guardados en carpetas separadas
- ☐ Puedo explicar cada proyecto a un entrevistador
- ☐ Entiendo el flujo completo: SQL Python Excel
- ☐ Estoy listo para el siguiente libro de la colección

Conclusión

Has completado el primer paso

Felicidades. Has completado "Los Fundamentos del Analista de Datos".

En este libro has aprendido:

SQL:

- Consultas básicas con SELECT, WHERE, ORDER BY
- Agregaciones con GROUP BY y HAVING
- Combinación de tablas con JOINS
- Subconsultas y CTEs
- Funciones de ventana (window functions)

Excel:

- Importación y transformación de datos
- Tablas dinámicas y segmentadores
- Visualización con gráficos profesionales
- Power Query para automatizar transformaciones
- Power Pivot y DAX para modelos de datos

Python:

- Variables, tipos de datos y operadores
- Estructuras de datos: listas, diccionarios
- Control de flujo: if, for, while
- Funciones y módulos
- pandas para análisis de datos

Integración:

- SQL + Python + Excel en un flujo de trabajo
- Pipeline automatizado de análisis
- Dashboard profesional con KPIs

¿Qué sigue?

Este libro es el Libro 1 de la colección **Data & Analytics Mastery**. Te recomiendo continuar con:

Libro 2: Python para Datos y tu Primer Data Warehouse Cloud

Pandas avanzado, BigQuery, Looker Studio

Libro 3: Business Intelligence y Modelado de Datos

Power BI, DAX avanzado, certificación PL-300

Libro 4: Ingeniería de Datos Moderna

Snowflake, dbt, Airflow

Libro 5: Big Data, ML e IA: Especialización Avanzada

Spark, streaming, machine learning, IA generativa

Último consejo

El análisis de datos no es un destino, es un viaje. Cada proyecto que hagas, cada error que cometas, cada pregunta que respondas te hará mejor analista.

Mantén la curiosidad. Sigue aprendiendo. Y sobre todo, sigue analizando.

Alex Goyzueta Delgado

Apéndice A: Soluciones a los Ejercicios

Capítulo 1: Ecosistema del Analista

1. 5 tablas: customers, employees, orders, order_items, products
2. 509 productos
3. 250 clientes
4. id, customer_id, employee_id, order_date, status, total
5. Desde 2023-01-01 hasta 2026-06-30
6. 25 empleados
7. 15 categorías
8. 6000 filas
9. .db es una base de datos relacional (SQLite), .csv es un archivo de texto plano con valores separados por comas
10. SQLite está optimizado para consultas en millones de filas, utiliza índices y no carga todo en memoria

Capítulo 2: SQL SELECT

1. ``SELECT * FROM customers LIMIT 10;``
2. ``SELECT first_name FROM customers;``
3. ``SELECT name, price FROM products ORDER BY price DESC LIMIT 5;``
4. ``SELECT COUNT(*) FROM products WHERE stock > 0;``
5. ``SELECT name, price FROM products ORDER BY price ASC LIMIT 3;``
6. ``SELECT first_name, last_name, position FROM employees ORDER BY hire_date;``
7. ``SELECT name AS Producto, price AS Precio FROM products;``
8. ``SELECT * FROM categories;``
9. ``SELECT id, order_date, total FROM orders ORDER BY order_date DESC LIMIT 10;``
10. 21118 filas

Capítulo 3: SQL WHERE

1. ``SELECT * FROM products WHERE price > 1000;``
2. ``SELECT COUNT(*) FROM products WHERE stock = 0;``
3. ``SELECT * FROM products WHERE category_id = 4 AND price < 100;``
4. ``SELECT * FROM customers WHERE region IN ('Madrid', 'Cataluña');``
5. ``SELECT * FROM orders WHERE total BETWEEN 100 AND 500 ORDER BY total DESC;``

6. ``SELECT COUNT(*) FROM orders WHERE status = 'Cancelado';`` (Aprox 450)
7. ``SELECT * FROM products WHERE name LIKE '%Ultra%';``
8. ``SELECT * FROM employees WHERE hire_date > '2024-01-01';``
9. ``SELECT COUNT(*) FROM products WHERE price IS NULL;`` (0)
10. ``SELECT name, price, stock FROM products WHERE stock > 0 ORDER BY price DESC LIMIT 10;``

Capítulo 4: Excel Importar

- 1-10: Soluciones prácticas. Los valores numéricos dependen de los datos. Verificar con fórmulas:
- SUMA, PROMEDIO, CONTAR, CONTAR.SI, SUMAR.SI

Capítulo 5: Python Variables

1. ``nombre_tienda = "TechStore"; print(nombre_tienda)``
2. ``ingresos = 1500000; gastos = 950000; print(f"Beneficio: {ingresos - gastos}€")``
3. ``nombre = input("¿Cómo te llamas? "); print(f"Hola {nombre}")``
4. ``precio = 499.99; print(f"Con IVA: {precio * 1.21:.2f}€")``
5. ``precio = float("1299.99"); print(precio + 200)``
6. ``from datetime import date; dias = (date.today() - date(2024, 1, 1)).days; print(dias)``
7. ``stock = 50; stock -= 3; print(f"Stock restante: {stock}")``
8. ``total = 25; mujeres = 12; print(f"Porcentaje: {mujeres/total*100:.1f}%")``
9. ``a = float(input("Número 1: ")); b = float(input("Número 2: ")); print(a + b)``
10. ``print(type(42), type(3.14), type("Hola"), type(True), type(None))``

Capítulo 6: SQL GROUP BY

1. ``SELECT status, COUNT(*) FROM orders GROUP BY status;``
2. ``SELECT c.region, ROUND(SUM(o.total), 2) FROM orders o JOIN customers c ON o.customer_id = c.id GROUP BY c.region;``
3. ``SELECT c.name, COUNT(*) FROM categories c JOIN products p ON c.id = p.category_id GROUP BY c.name;``
4. ``SELECT city, COUNT() FROM customers GROUP BY city ORDER BY COUNT() DESC;``
5. ``SELECT c.region, ROUND(AVG(o.total), 2) FROM orders o JOIN customers c ON o.customer_id = c.id GROUP BY c.region;``
6. ``SELECT customer_id, COUNT() FROM orders GROUP BY customer_id HAVING COUNT() > 5;``
7. Ver consulta 3.1 del Checkpoint 2
8. ``SELECT strftime('%Y-%m', order_date), COUNT(*) FROM orders WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01' GROUP BY 1;``
10. ``SELECT c.name, ROUND(AVG(p.price), 2) FROM categories c JOIN products p ON c.id = p.category_id GROUP BY c.name ORDER BY 2 DESC;``

Capítulo 7: SQL JOINS

3. ``SELECT p.name, c.name AS categoria, p.price FROM products p JOIN categories c ON p.category_id = c.id ORDER BY p.price DESC;``
5. ``SELECT p.name, ROUND(SUM(oi.quantity * oi.unit_price), 2) FROM products p JOIN order_items oi ON p.id = oi.product_id JOIN orders o ON oi.order_id = o.id WHERE o.status = 'Completado' GROUP BY p.id ORDER BY 2 DESC;``
6. ``SELECT * FROM customers c LEFT JOIN orders o ON c.id = o.customer_id WHERE o.id IS NULL;``
8. ``SELECT c.name, MAX(p.price) AS caro, MIN(p.price) AS barato FROM categories c JOIN products p ON c.id = p.category_id GROUP BY c.name;``
9. Usar COUNT(DISTINCT oi.product_id) con JOINS
10. Ver Checkpoint 2

Capítulo 8: Excel Tablas Dinámicas

Soluciones prácticas: crear tabla dinámica, arrastrar campos a filas/columnas/valores, cambiar tipo de cálculo.

Capítulo 9: Python Estructuras

1. ``productos = ["Portátil", "Smartphone", "Tablet", "Auriculares", "Ratón"]``
2. ``productos.append("Teclado"); productos.append("Monitor"); productos.append("Altavoz")``
3. ``cliente = {"nombre": "Ana", "email": "ana@email.com", "ciudad": "Madrid", "gasto_total": 1250.50}``
4. ``cliente["gasto_total"] += 150``
5. ``[round(p * 1.21, 2) for p in [299, 599, 899, 1299, 1999]]``
6. ``cesta = [("Portátil", 1299.99), ("Ratón", 29.99), ("Teclado", 89.99)]; total = sum(p * c for _, p in cesta)``
7. ``precios = [100, 500, 50, 1000, 200, 800]; [p for p in precios if p > 500]``
8. Usar diccionario y bucle para restar 1 a cada stock
9. ``max(productos, key=lambda p: p["precio"])``
10. ``def analisis(ventas): return {"promedio": sum(ventas)/len(ventas), "max": max(ventas), "min": min(ventas)}``

Capítulo 10: Excel Gráficos

Soluciones prácticas: insertar gráfico, seleccionar datos, personalizar.

Capítulo 11: SQL Subconsultas

1. ``SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products);``
2. ``SELECT customer_id, SUM(total) FROM orders GROUP BY customer_id HAVING SUM(total) > (SELECT AVG(total) FROM orders);``
3. ``SELECT , ROUND(total / (SELECT SUM(total) FROM orders WHERE status = 'Completado') 100, 2) FROM orders WHERE status = 'Completado';``
4. ``WITH ventas_mes AS (SELECT strftime('%Y-%m', order_date) AS mes, SUM(total) AS`

ingresos FROM orders WHERE status = 'Completado' GROUP BY mes) SELECT * FROM ventas_mes WHERE ingresos > 100000;`

5. Crear CTEs y calcular porcentajes

6. `SELECT employee\_id, COUNT() FROM orders GROUP BY employee\_id HAVING COUNT() > (SELECT AVG(cnt) FROM (SELECT COUNT(\*) AS cnt FROM orders GROUP BY employee\_id));`

7. Con subconsulta correlacionada comparando stock con AVG(stock) de su categoría

8. CTE con ROW_NUMBER() PARTITION BY category_id

9. Crear CTEs para cada año y comparar

10. Con NOT EXISTS y subconsulta

Capítulo 12: Window Functions

1. `SELECT name, price, ROW\_NUMBER() OVER (ORDER BY price DESC) FROM products;`

2. `SELECT customer\_id, gasto, NTILE(5) OVER (ORDER BY gasto) FROM (SELECT customer\_id, SUM(total) AS gasto FROM orders GROUP BY customer\_id);`

3. `SELECT mes, ingresos, SUM(ingresos) OVER (ORDER BY mes) FROM ventas\_mensuales;`

4. Con FIRST_VALUE o MAX() OVER (PARTITION BY category_id)

5. `SELECT mes, ingresos, LAG(ingresos) OVER (ORDER BY mes) FROM ventas\_mensuales;`

6. Con ROW_NUMBER() PARTITION BY category_id ORDER BY unidades DESC

7. Con AVG(total) OVER (ORDER BY fecha ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)

8. `SELECT mes, ingresos, LEAD(ingresos) OVER (ORDER BY mes) FROM ventas\_mensuales;`

9. `SELECT order\_id, customer\_id, total, ROUND(total / SUM(total) OVER (PARTITION BY customer\_id, 4) FROM orders WHERE status = 'Completado';`

10. `SELECT \*, price > AVG(price) OVER (PARTITION BY category\_id) AS sobre\_promedio FROM products;`

Capítulo 13: Power Query

Soluciones prácticas en Power Query Editor.

Capítulo 14: Python Funciones

1. `def calcular\_iva(precio, tasa=0.21): return round(precio \* (1 + tasa), 2)`

2-10: Ver archivos de soluciones en codigos/soluciones/

Capítulo 15: DAX

Ver medidas DAX en el capítulo 17 para soluciones completas.

Apéndice B: Cheatsheets

SQL Cheatsheet

Consultas básicas

```
SELECT col1, col2 FROM tabla;  
SELECT * FROM tabla LIMIT 10;  
SELECT DISTINCT columna FROM tabla;
```

Filtros

```
WHERE condicion  
WHERE col > 100 AND col < 500  
WHERE col IN ('A', 'B', 'C')  
WHERE col BETWEEN 100 AND 500  
WHERE col LIKE '%patron%'  
WHERE col IS NULL  
WHERE col IS NOT NULL
```

Ordenación

```
ORDER BY col ASC      -- Ascendente (por defecto)  
ORDER BY col DESC     -- Descendente  
ORDER BY col1, col2 -- Múltiples columnas
```

Agregación

```
GROUP BY col  
HAVING condicion  -- Filtrar grupos  
COUNT(*), SUM(col), AVG(col), MIN(col), MAX(col)
```

JOINS

```
JOIN tabla2 ON t1.col = t2.col      -- INNER JOIN  
LEFT JOIN tabla2 ON t1.col = t2.col -- LEFT JOIN  
RIGHT JOIN tabla2 ON t1.col = t2.col -- RIGHT JOIN
```

Subconsultas y CTEs

```
SELECT * FROM tabla WHERE col IN (SELECT col FROM otra);  
WITH cte AS (SELECT ...) SELECT * FROM cte;
```

Window Functions

```
ROW_NUMBER() OVER (PARTITION BY col ORDER BY col)
RANK() OVER (ORDER BY col)
DENSE_RANK() OVER (ORDER BY col)
SUM(col) OVER (PARTITION BY col ORDER BY col)
LAG(col) OVER (ORDER BY col)
LEAD(col) OVER (ORDER BY col)
NTILE(n) OVER (ORDER BY col)
```

Excel Cheatsheet

Atajos de teclado

Atajo	Acción
Ctrl+T	Crear tabla
Ctrl+Shift+L	Activar filtro
Ctrl+Shift+1	Formato número
Ctrl+Shift+4	Formato moneda
Ctrl+Shift+5	Formato porcentaje
F4	Repetir última acción
Ctrl+;	Fecha actual
Alt+=	Auto suma

Fórmulas esenciales

```
=SUMA(rango)
=PROMEDIO(rango)
=CONTAR(rango)
=CONTARA(rango)
=CONTAR.SI(rango; criterio)
=SUMAR.SI(rango_criterio; criterio; rango_suma)
=SI(condicion; valor_si; valor_no)
=BUSCARX(valor; rango_busqueda; rango_resultado)
```

Power Query - Transformaciones

- Promover encabezados
- Cambiar tipo de datos
- Eliminar filas/columnas
- Reemplazar valores
- Dividir columna
- Combinar consultas (Merge)
- Agrupar por
- Columna personalizada

Python Cheatsheet

Tipos de datos

```
int, float, str, bool, None, list, dict, tuple
```

Operadores

```
+ - * / // % **    # Aritméticos  
== != > < >= <=   # Comparación  
and or not         # Lógicos
```

Strings

```
f"texto {variable}" # f-strings  
len("texto")  
"texto".upper(), .lower(), .strip()
```

Listas

```
lista = [1, 2, 3]  
lista.append(4)  
lista[0], lista[-1], lista[1:3]  
len(lista), sum(lista), max(lista), min(lista)  
[expr for item in lista if cond] # List comprehension
```

Diccionarios

```
dic = {"clave": "valor"}  
dic["clave"], dic.get("clave")  
dic.keys(), dic.values(), dic.items()  
for k, v in dic.items(): ...
```

Control de flujo

```
if cond: ... elif: ... else: ...  
for item in lista: ...  
for i in range(n): ...  
while cond: ...
```

pandas

```
import pandas as pd  
df = pd.read_csv("archivo.csv")  
df = pd.read_sql("query", conn)  
df.head(), df.shape, df.dtypes  
df[df["col"] > 100] # Filtro  
df.groupby("col").agg({"val": "sum"}) # Agregación  
df.sort_values("col", ascending=False)  
df.to_excel("output.xlsx", index=False)
```

Power Pivot / DAX Cheatsheet

```
-- Medidas básicas
Ventas = SUM(tabla[columna])
Pedidos = COUNTROWS(tabla)
Clientes = DISTINCTCOUNT(tabla[columna])
Promedio = AVERAGE(tabla[columna])

-- CALCULATE
CALCULATE(medida, condición)

-- Iteración
SUMX(tabla, expresión)
AVERAGEX(tabla, expresión)

-- Tiempo
TOTALYTD(medida, calendario[fecha])
SAMEPERIODLASTYEAR(calendario[fecha])
DIVIDE(numerador, denominador)
```

Apéndice C: Instalación y Configuración

1. DB Browser for SQLite

Descarga: <https://sqlitebrowser.org/dl/>

Instalación

1. Descarga el instalador para Windows (portable o installer)
2. Ejecuta el instalador
3. Sigue los pasos del asistente
4. Abre DB Browser y selecciona "Open Database"
5. Navega a `codigos/techstore.db`

Verificación

- Ejecuta `SELECT COUNT(\*) FROM products;` debe devolver 509

2. Python 3.12+

Descarga: <https://www.python.org/downloads/>

Instalación

1. Descarga Python 3.12 o superior
2. **IMPORTANTE:** marca "Add Python to PATH"
3. Haz clic en "Install Now"
4. Espera a que termine la instalación

Verificación

Abre una terminal (cmd) y escribe:

```
python --version
pip --version
```

Instalar paquetes

```
pip install pandas openpyxl
```

3. Visual Studio Code

Descarga: <https://code.visualstudio.com/>

Instalación

1. Descarga el instalador
2. Ejecuta y sigue los pasos
3. Abre VS Code

Extensiones recomendadas

- **Python** (Microsoft) — Soporte para Python
- **SQLite Viewer** — Ver bases de datos SQLite
- **Excel Viewer** — Vista previa de CSV
- **Spanish Language Pack** — Interfaz en español (opcional)
- **Markdown Preview** — Vista previa de markdown

Configurar VS Code para Python

1. Abre VS Code
2. Ctrl+Shift+P > "Python: Select Interpreter"
3. Selecciona la versión de Python instalada
4. Crea un archivo `.py`` y escribe `print("Hola")``
5. Haz clic en "Run" (triángulo en esquina superior derecha)

4. Microsoft Excel

Si no tienes Excel

Opciones gratuitas compatibles:

- **LibreOffice Calc**: <https://www.libreoffice.org/>
- **Google Sheets**: sheets.google.com (gratuito con cuenta Google)
- **OnlyOffice**: <https://www.onlyoffice.com/>

Power Pivot (solo Excel)

1. Archivo > Opciones > Complementos
2. En "Administrar", selecciona "Complementos COM"
3. Marca "Microsoft Power Pivot for Excel"
4. Aparecerá la pestaña "Power Pivot" en la cinta

Power Query (Excel 2016+)

Ya viene incluido en "Datos" > "Obtener y transformar datos"

5. Estructura de archivos del libro

```
fundamentos_analista_datos/  
├── codigos/  
│   ├── techstore.db          # Base de datos SQLite  
│   ├── datos_ventas.csv      # 6000 pedidos  
│   ├── datos_clientes.csv    # 250 clientes  
│   ├── datos_productos.csv   # 509 productos  
│   ├── datos_detalle_ventas.csv # 21118 líneas  
│   ├── ejemplo_1.sql         # SQL de ejemplo  
│   └── ejemplo_2.sql
```

```

|   ├── ejemplo_3.sql
|   └── generar_db_techstore.py  # Script para regenerar la BD
├── *.md                        # Capítulos del libro
├── metadata.yaml              # Metadatos del libro
├── cover.jpg                  # Portada
├── descripcion.txt            # Descripción
└── toc.md                     # Tabla de contenidos

```

6. Solución de problemas comunes

Python no se reconoce como comando

- Solución: reinstala Python marcando "Add Python to PATH"

• O: usa la ruta completa:
`C:\Users\tu\_usuario\AppData\Local\Programs\Python\Python312\python.exe`

pandas no encontrado

- Abre terminal y ejecuta: `pip install pandas`

SQLite no abre la base de datos

- Verifica que `techstore.db` existe en la carpeta correcta
- Si no existe, ejecuta: `python codigos/generar\_db\_techstore.py`

Excel no abre CSV correctamente

- Usa "Datos" > "Desde archivo" > "Desde CSV" en lugar de doble clic
- Selecciona el separador correcto (coma) y codificación UTF-8

Power Query no aparece

- Solo disponible en Excel 2016 o superior
- Verifica que no estás en modo de compatibilidad (.xls en lugar de .xlsx)

7. Recursos online

- Repositorio GitHub del libro: [URL]
- Códigos y soluciones: carpeta `codigos/soluciones/`
- Comunidad Data & Analytics Mastery: [URL]
- Canal de YouTube con videotutoriales: [URL]

Sobre el Autor

Alex Goyzueta Delgado es analista de datos, escritor técnico y divulgador de tecnología. Con experiencia en SQL, Excel, Python, Power BI y herramientas cloud, ha ayudado a cientos de estudiantes a iniciar su carrera en el mundo de los datos.

Cree firmemente en que la mejor forma de aprender es haciendo. Por eso todos sus libros combinan teoría con proyectos prácticos basados en casos reales.

Es autor de la colección **Data & Analytics Mastery**, una serie de 5 libros que cubren desde los fundamentos del análisis de datos hasta la especialización en big data, machine learning e inteligencia artificial.

También ha escrito más de 15 libros técnicos en formato de novela, incluyendo la exitosa serie de "SQL Letal", "Excel Básico/Intermedio/Avanzado" y "Código Asesino", todos disponibles en formato EPUB y PDF.

Contacto:

- Email: alexgoyzueta2018@gmail.com
- Payhip: <https://payhip.com/SistGoy>

Colección Data & Analytics Mastery:

1. Los Fundamentos del Analista de Datos *Este libro*
2. Python para Datos y tu Primer Data Warehouse Cloud
3. Business Intelligence y Modelado de Datos
4. Ingeniería de Datos Moderna
5. Big Data, ML e IA: Especialización Avanzada

"Los datos no son solo números. Son historias esperando ser contadas."